Chapter 12: IPSec

Introduction

- Repitition of the Internet Protocol (IP) Suite
- Security problems of IP and objectives of IPSec

IPSec Architecture

- Overview
- IPSec Replay Protection
- IPSec Security Protocol Modes
- Security Policies
- Security Associations
- Processing of IPSec Traffic

IPSec Security Protocols

- Encapsulating Security Payload (ESP)
- Authentication Header (AH)
- Cryptographic Algorithms in IPSec

Chapter 12: IPSec

Entity Authentication and Key Exchange

- Introduction
- Protocol Exchanges
- Generation of Keying Material
- Negotiation of Security Associations
- Advanced IKEv2 Features
- Message and Payload Format
- Summary
- Outlook

WireGuard

Chapter 12: IPSec

Introduction

- Repitition of the Internet Protocol (IP) Suite
- Security problems of IP and objectives of IPSec

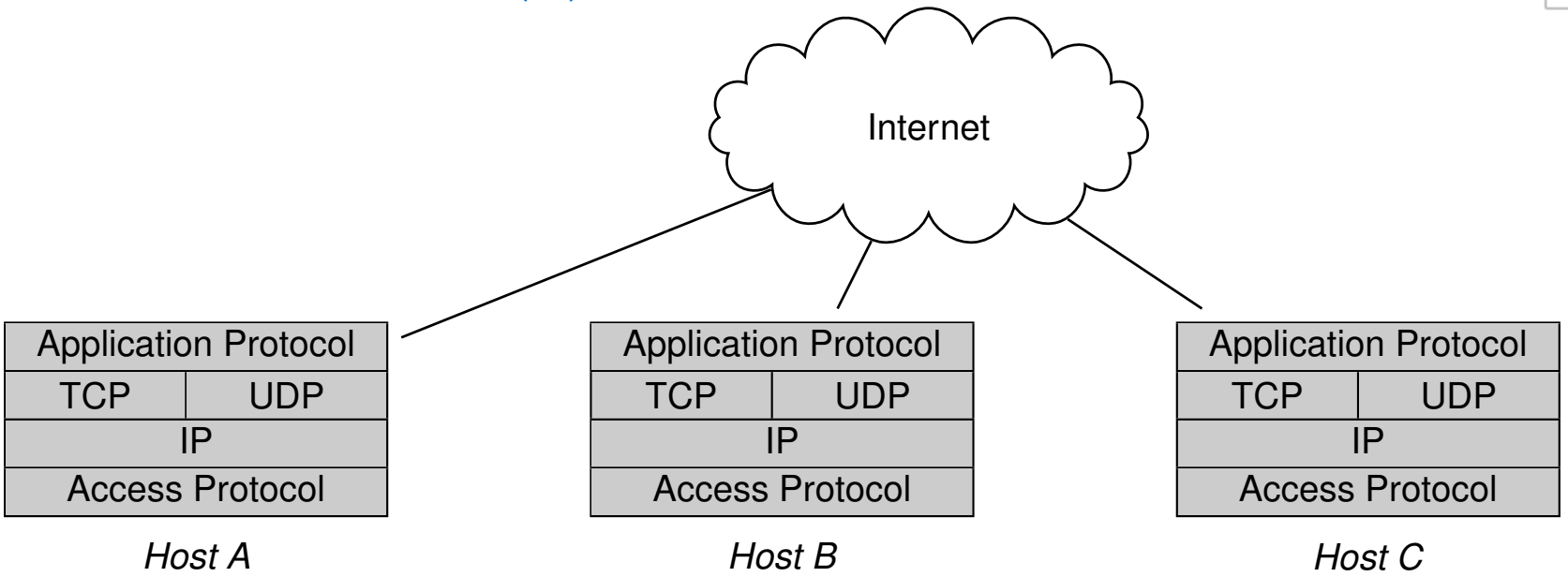
IPSec Architecture

IPSec Security Protocols

Entity Authentication and Key Exchange

WireGuard

Repitition of the Internet Protocol (IP) Suite



IP (Internet Protocol)

- unreliable, connectionless network protocol

TCP (Transmission Control Protocol)

- reliable, connection-oriented transport protocol

UDP (User Datagram Protocol)

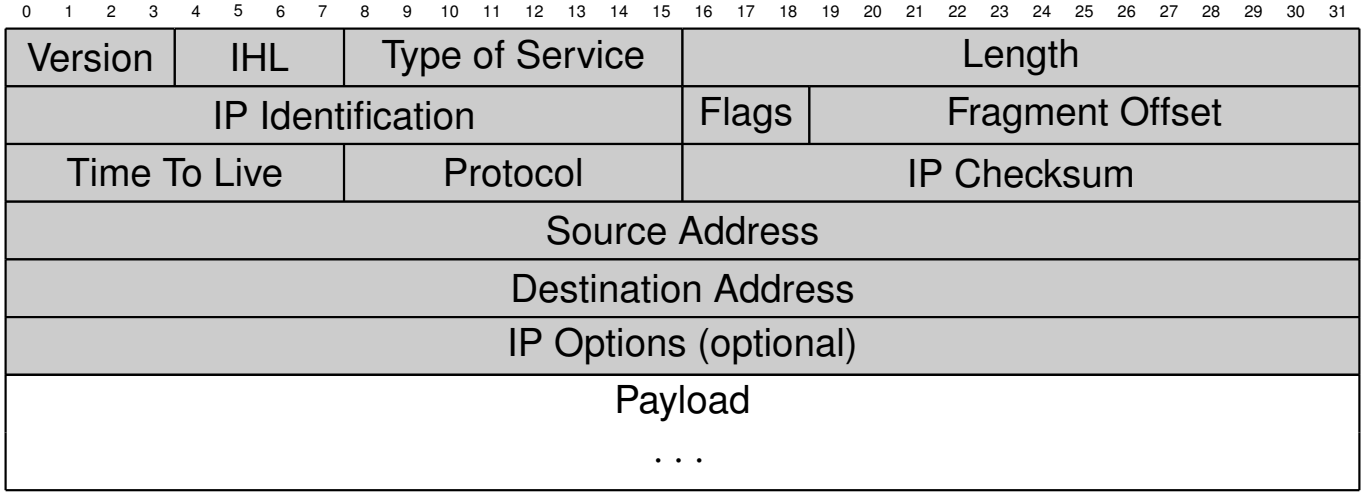
- unreliable, connectionless transport protocol

Application Protocols

- E.g. *HTTP* (Hypertext Transfer Protocol), *SMTP* (Simple Mail Transfer Protocol), ...

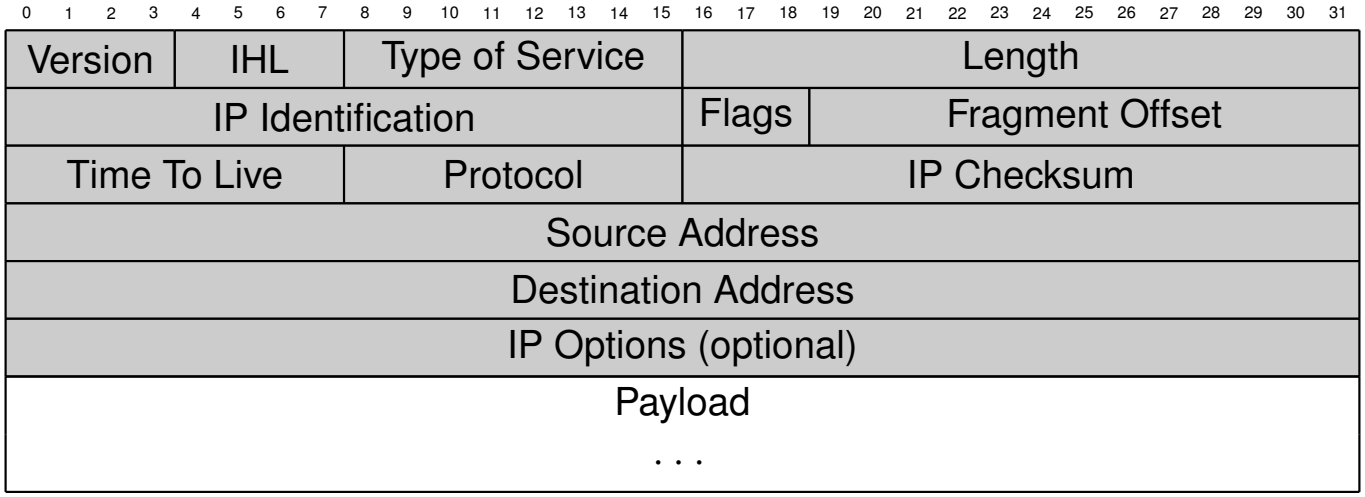
Repitition of the Internet Protocol (IP) Suite

IPv4 Packet Format



- Version:
  - Currently version 4 is widely deployed
  - Version 6 is gradually taking over
- Internet Header Length (IHL):
  - Length of the header in 32bit words
  - I.e.: No options $\Rightarrow$ IHL = 5
- Type of Service (ToS):
  - Originally to indicate service requirement
  - Redefined for service differentiation and Explicit Congestion Notification
- Length:
  - Length of the packet including header in octets
  - Field given in *Big Endian* (like all other IP fields)
- Identificiation:
  - Used to "uniquely" identify IP datagram
  - For reassembling fragmented datagrams
- Flags:
  - Bit 1: *Do not fragment*
  - Bit 2: *Datagram fragmented*
  - Bit 3: *Reserved*

IPv4 Packet Format



- Fragmentation offset:
  - Position of this packet in IP datagram
- Time to live (TTL):
  - Every relaying node decrements this by one
  - When $TTL = 0$ the packet is discarded
- Protocol:
  - Indicates (transport) protocol of the payload
  - Examples: TCP, UDP, ...
- Checksum:
  - Protects header against transmission errors
  - NOT a cryptographic checksum
- Source Address:
  - IP adress of the sender
- Destination Address:
  - IP adress of the receiver
- IP Options:
  - IP headers can optionally carry additional information

Security problems of IP and objectives of IPSec

On arrival of an IP packet there is no assurance of

1. Data origin authentication
   - The packet was actually sent by the entity referenced by the source address
2. Data Integrity
   - The original packet content was not modified during transit
3. Confidentiality
   - The packet data was not inspected by a third party during transit

⇑ IPSec tries to adress these issues ⇑

Security problems of IP and objectives of IPSec

If IPSec is correctly employed, it provides

1. Data origin authentication
   - It is not possible to spoof source / destination addresses without the receiver being able to detect this
   - It is not possible to replay a recorded IP packet without the receiver being able to detect this
2. Connectionless Data Integrity
   - It is not possible to modify an IP datagram in transit, without the receiver being able to detect this
3. Confidentiality
   - It is not possible to eavesdrop on the content of IP datagrams
   - Limited traffic flow confidentiality
4. Security Policies
   - All involved nodes can determine the required protection for a packet according to a local security policy
   - Intermediate nodes and the receiver will drop packets, that don't meet these requirements

Chapter 12: IPSec

Introduction

IPSec Architecture

- Overview
- IPSec Replay Protection
- IPSec Security Protocol Modes
- Security Policies
- Security Associations
- Processing of IPSec Traffic

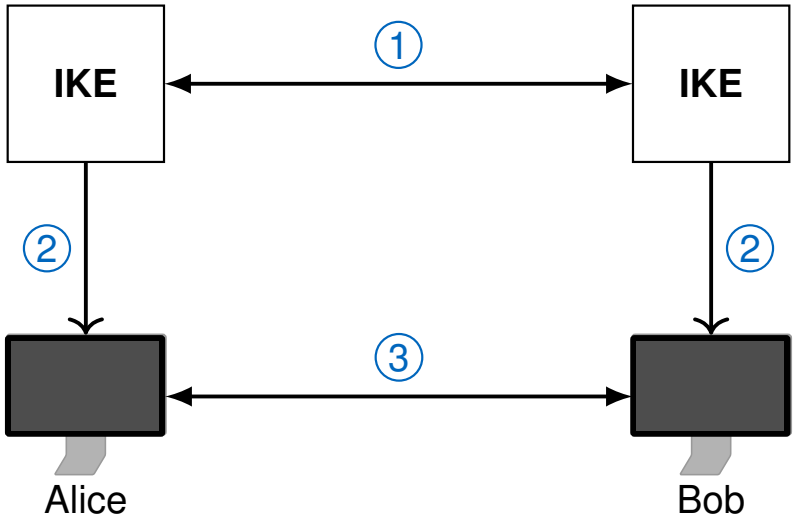
IPSec Security Protocols

Entity Authentication and Key Exchange

WireGuard

Overview

How does IPSec work?

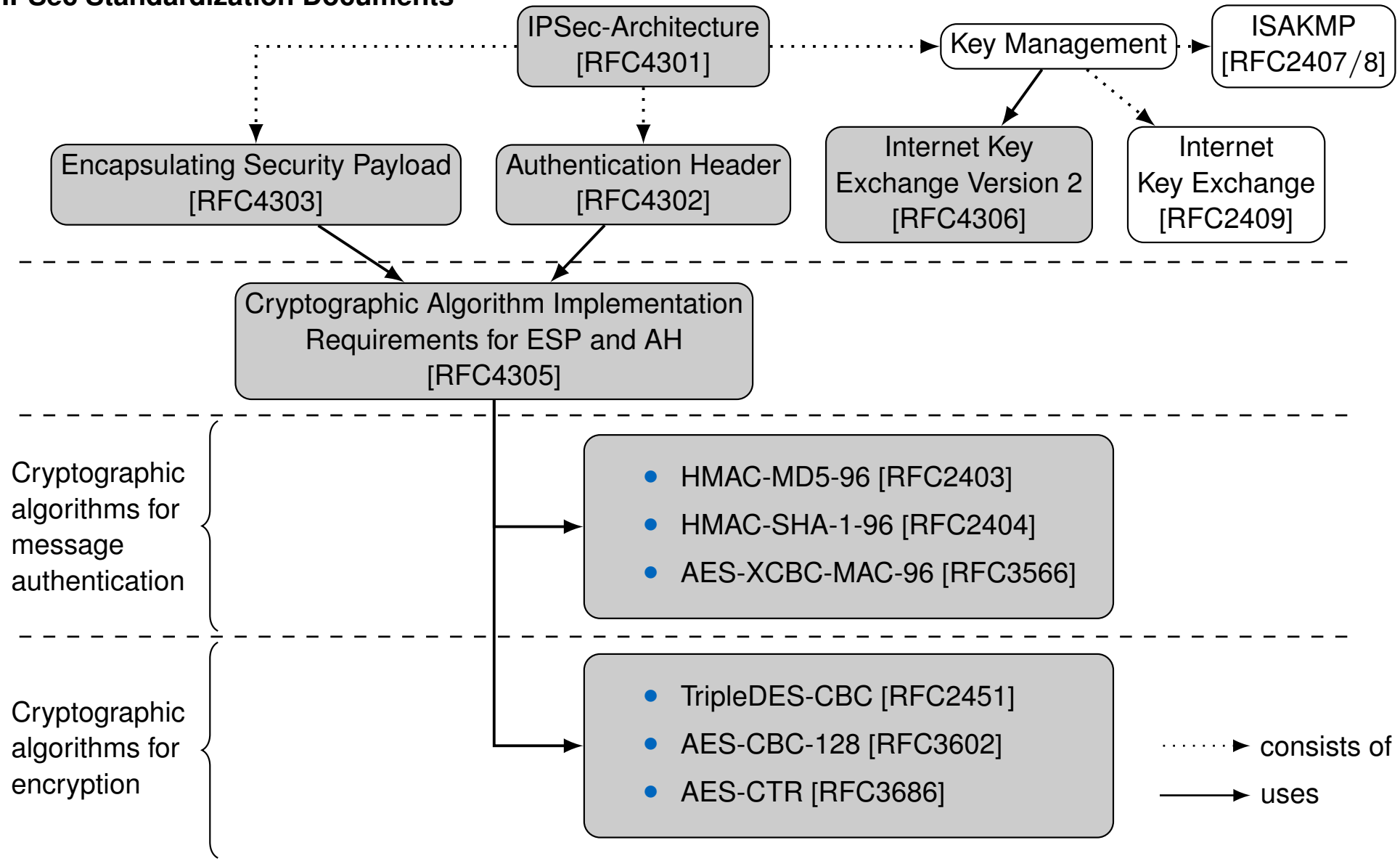


1. Authentication, key establishment and negotiation of crypto algorithms
   - Possible protocols: ISAKMP, Internet Key Exchange (IKE), IKEv2
2. Set keys and cryptographic algorithms
3. Secure channel which provides
   - Data Integrity via Authentication Header (AH) or Encapsulating Security Payload (ESP)
   - Confidentiality using ESP

Note: ESP can provide both data integrity and encryption while AH only provides data integrity

Overview

IPSec Standardization Documents



Overview

RFC 4301 defines the basic architecture of IPSec:

- Concepts
  - Security Association (SA) and Security Association Database (SAD)
  - Security Policy (SP) and Security Policy Database (SPD)
- Fundamental Protocols
  - Authentication Header (AH)
  - Encapsulation Security Payload (ESP)
- Protocol Modes
  - Transport Mode
  - Tunnel Mode
- Key Management Protocols
  - ISAKMP, IKE, IKEv2

List of IPSec related RFCs: https://datatracker.ietf.org/wg/ipsec/documents/

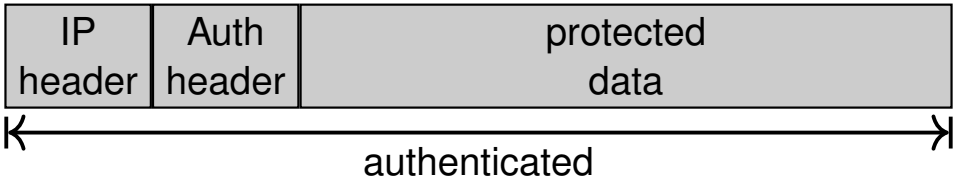
- Most RFCs updated in 2005 after several years of revision
- Support for integration of new crypto primitives for encryption and integrity
- Reduced complexity and better protocol design

Overview

Available Protocols

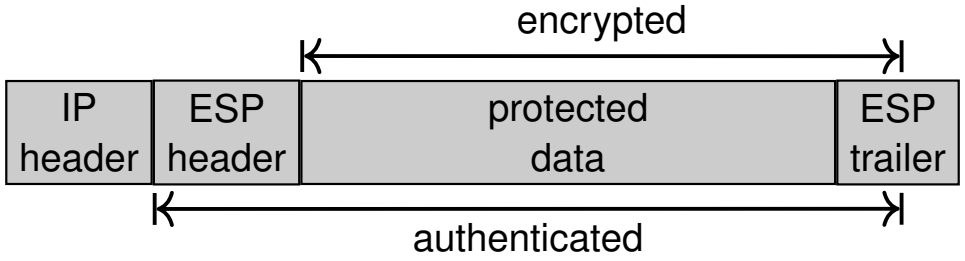
Authentication Header (AH):

- Data origin authentication and replay protection
- Inserted between the IP header and the data to be protected



Encapsulating Security Payload (ESP):

- Data origin authentication, confidentiality and replay protection
- A header and a trailer encapsulating the data to be protected



Overview

Key management and setup of Security Associations (SA)

Internet Security Association Key Management Protocol (ISAKMP)

- Defines generic framework for authentication, key exchange and SA parameters [RFC2408]
- Does not define a specific authentication protocol but defines
  - Packet formats
  - Retransmission formats
  - Message construction requirements
- Use of ISAKMP for IPSec is further described in [RFC2407]

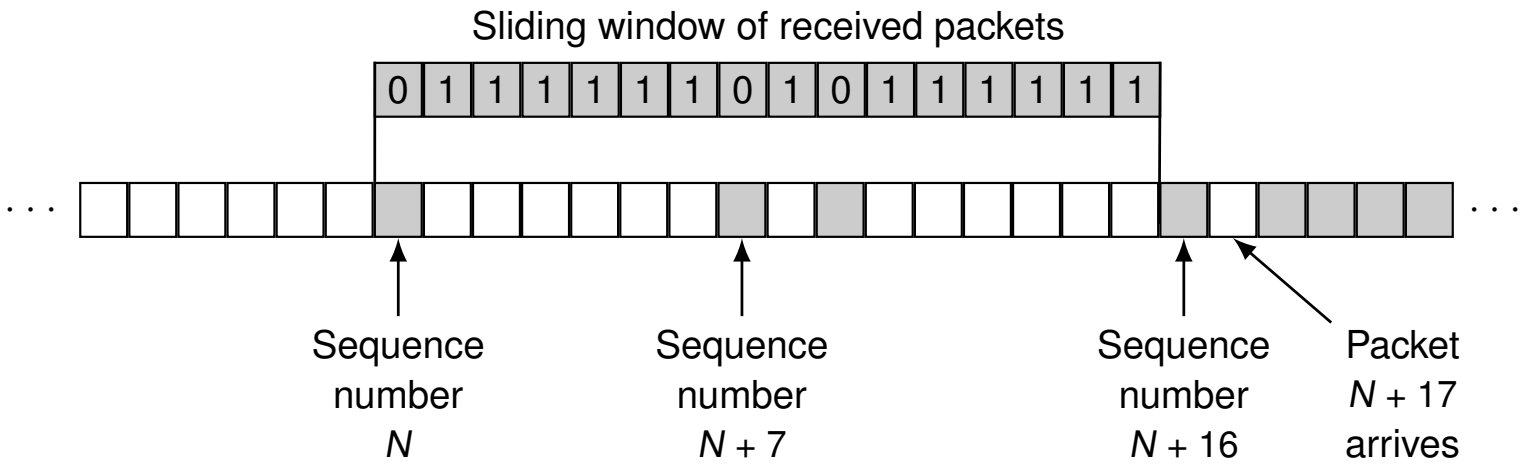
Internet Key Exchange Version 2 [RFC4306]

- Defines an authentication and key exchange protocol
- Reduced complexity by better protocol design and by omitting unnecessary features

IPSec Replay Protection

AH- and ESP-protected packets carry a sequence number

- On setup of a Security Association (SA) this sequence number is initialized to zero
- The sequence number is increased with every IP packet sent
- The sequence number is 32-bit long and a new session key is needed before a wrap-around occurs
- The receiver checks if the sequence number is contained in a window of acceptable numbers



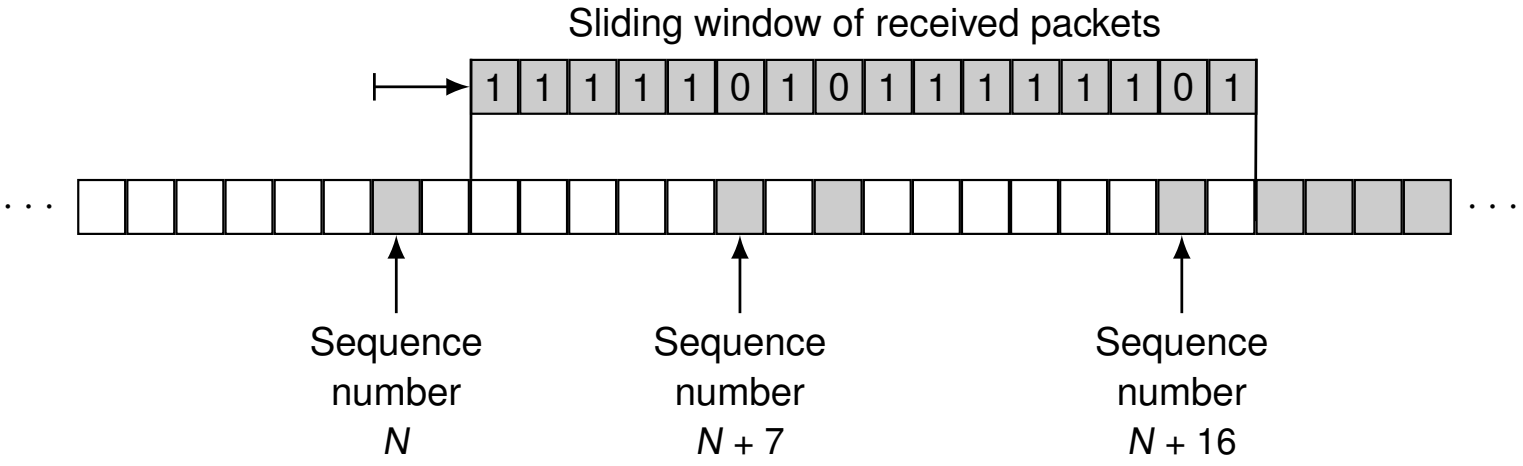
- Packet with sequence number *N* can still be accepted
- Window size has to be at least 32 in practice

IPSec Replay Protection

If a received packet has a sequence number which

- is left of the current window ⇒ the receiver rejects the packet
- is inside the current window ⇒ the receiver accepts the packet
- is right of the current window ⇒ the receiver accepts the packet

and advances the window.



- Packet with sequence number N can no longer be accepted
- IP packets are only accepted after successful authentication verification
- The window is never advance before this verification

IPSec Security Protocol Modes

Transport Mode

- Only usable between communication endpoints
  - Host $\leftrightarrow$ Host
  - Host $\leftrightarrow$ Gateway (e.g. Gateway is end-point for network management)
- Adds a security specific header (+ trailer if ESP is employed)

IP header	IPSec header	protected data

Tunnel Mode

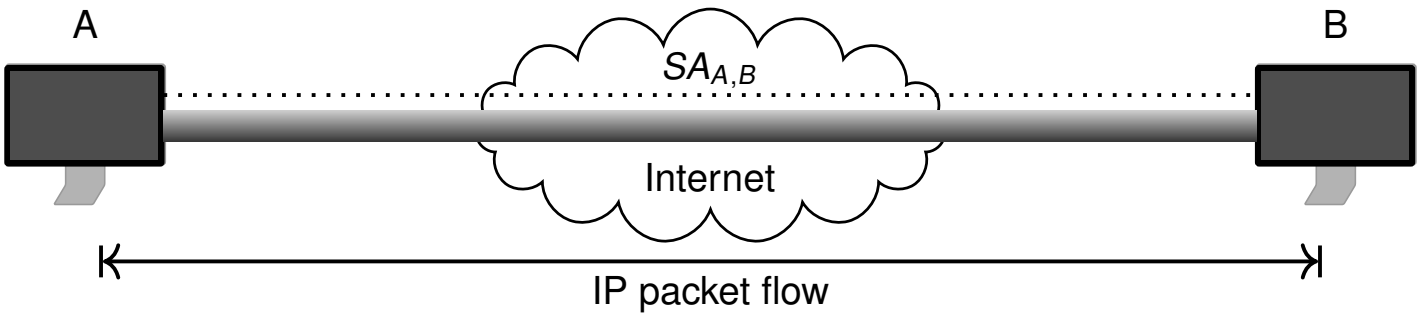
- Usable with arbitrary peers
- Encapsulates IP packets

IP header	IPSec header	IP header	protected data

- Allows for e.g. a gateway, protecting traffic on behalf of hosts in subnetwork

Transport Mode

- Used when the *cryptographic endpoints* are also the *communication endpoints* of the secured packets
  - Cryptographic Endpoints: Entities that process IPSec headers
  - Communication Endpoints: Source and destination of an IP packet

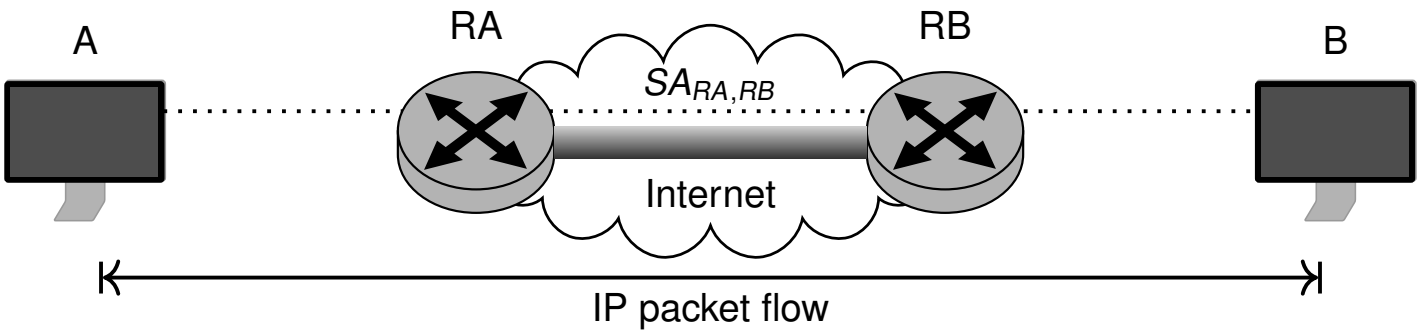


- In most cases, *communication endpoints* are hosts
- But not always the case (e.g. Gateway being managed by SNMP)

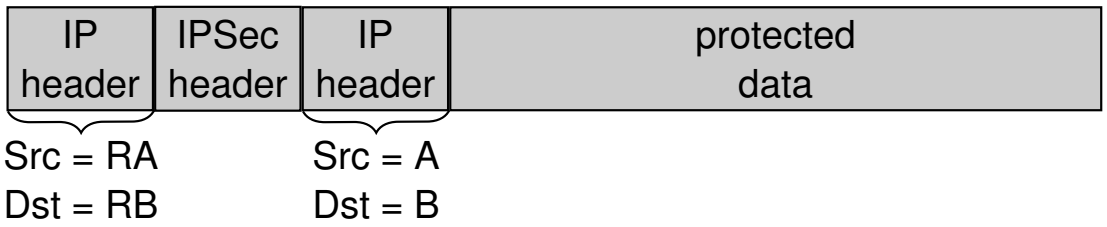
IPSec Security Protocol Modes

Tunnel Mode

- Used when at least one *cryptographic endpoint* is not a *communication endpoint*.
- This allows for gateways securing IP traffic on behalf of other entities



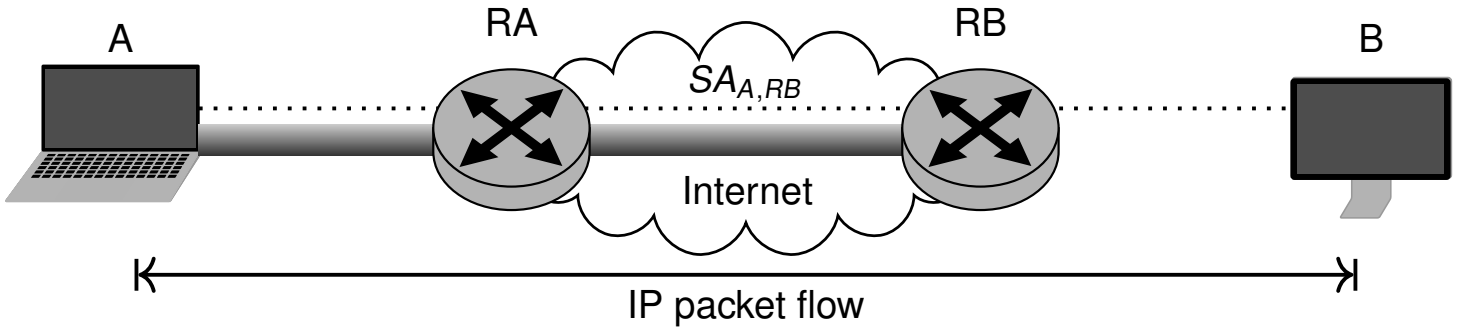
Packet Structure



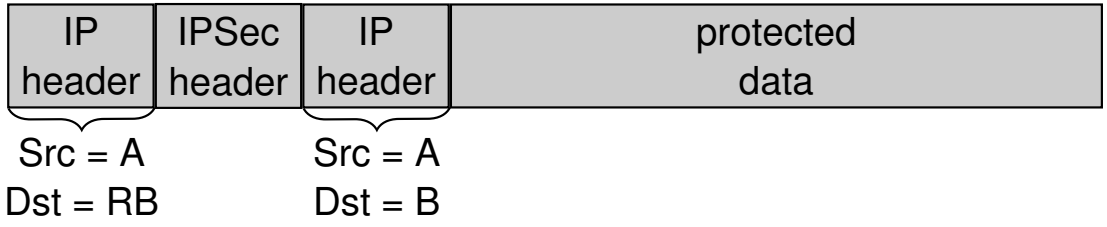
What if only one cryptographic endpoint is a communication endpoint?

Tunnel Mode

- Used when at least one *cryptographic endpoint* is not a *communication endpoint*.
- Example: Security gateway, ensuring authentication and/or confidentiality between subnetwork and host



Packet Structure



Traffic Selectors

A Traffic Selector (TS) is a set of properties used to characterize IP packets. Each TS may contain:

- IP source address
  - Specific host, network prefix, address range or wildcard
- IP destination address
  - Specific host, network prefix, address range or wildcard
  - In case of incoming tunneled packets the inner header is evaluated
- Name
  - DNS name, X.500 name or other name types
- Protocol
  - Protocol identifier of the transport protocol for this packet (e.g. TCP/UDP)
  - This may not be accessible when a packet is secured with ESP

Traffic Selectors are used to define Security Policies!

Security Policies

Definition

A Security Policy (SP) specifies which and how security services should be provided to IP packets.

This includes

- Selectors that identify specific IP flows
- Required security attributes for each flow
  - Security protocol (AH / ESP)
  - Protocol Mode (Transport / Tunnel)
  - Other parameters (e.g. policy lifetime, port number, ...) $\Leftarrow$ IPSec protection can be specified for specific applications!
- Actions (e.g. Discard, Secure, Bypass)

Security Policies are stored in the Security Policy Database (SPD)

Security Associations

Definition

A Security Association (SA) is a simplex channel that describes the way how packets need to be processed

As such

- An SA defines employed encryption / authentication algorithms and keys
- An SA is associated with either AH or ESP but not both
- Bidirectional communication requires two security associations
- An SA can be setup as
  - Host $\leftrightarrow$ Host
  - Host $\leftrightarrow$ Gateway
  - Gateway $\leftrightarrow$ Gateway

Security Associations are stored in the Security Association Database (SAD)

Security Associations

In the Security Association Database (SAD)

- an entry (SA) is uniquely identified by a Security Parameter Index (SPI)
- the SPI value is specified by the receiving side during SA negotiation
- the SPI value for construction of AH/ESP headers is looked up for outbound SAs
- the SPI value is used to map the traffic to the appropriate SA for inbound traffic

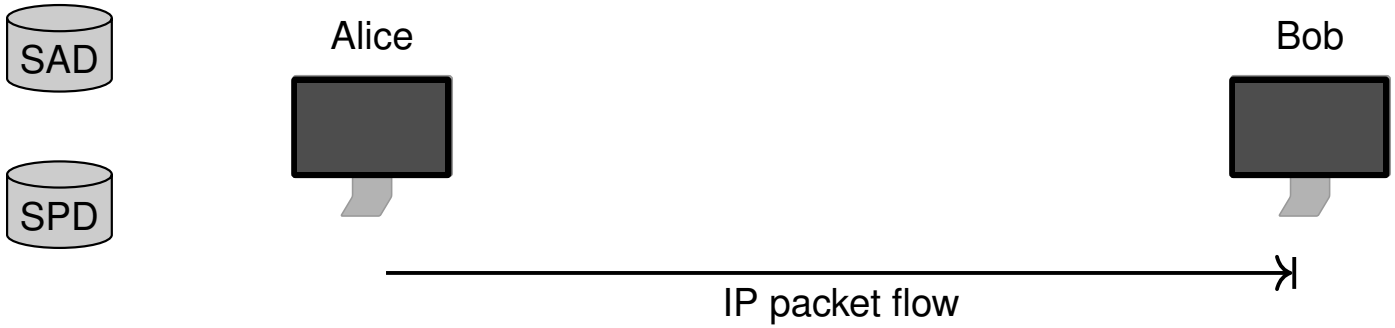
An SA entry in the SAD includes

- Security Parameter Index (SPI)
- IP source / destination address
- A security protocol identified (AH / ESP)
- Current sequence number counter (replay protection)
- Protocol algorithms, modes, IVs and keys for authentication and encryption
- Security Association Lifetime
- IPSec protocol mode (tunnel / transport)
- Additional information (see RFC4301, Section 4.4.2.1)

Now, how does this work in practice?

Processing of IPSec Traffic

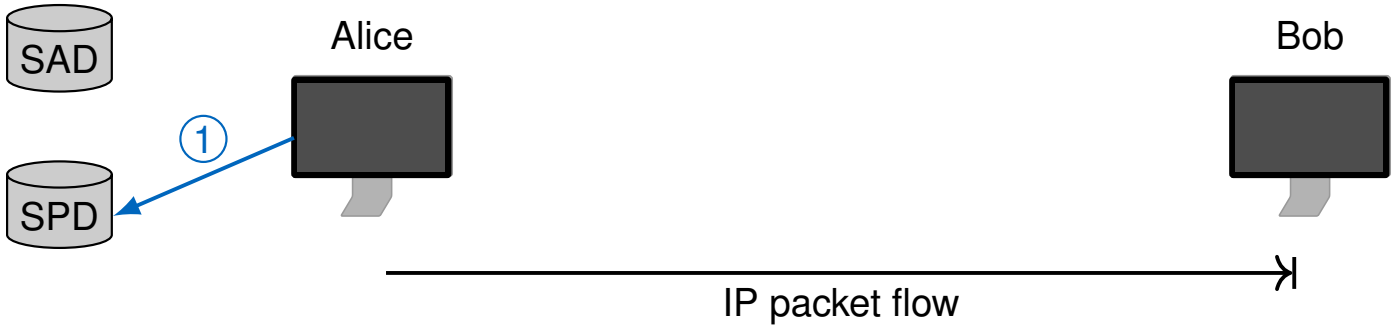
Outgoing Traffic



Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

Processing of IPSec Traffic

Outgoing Traffic

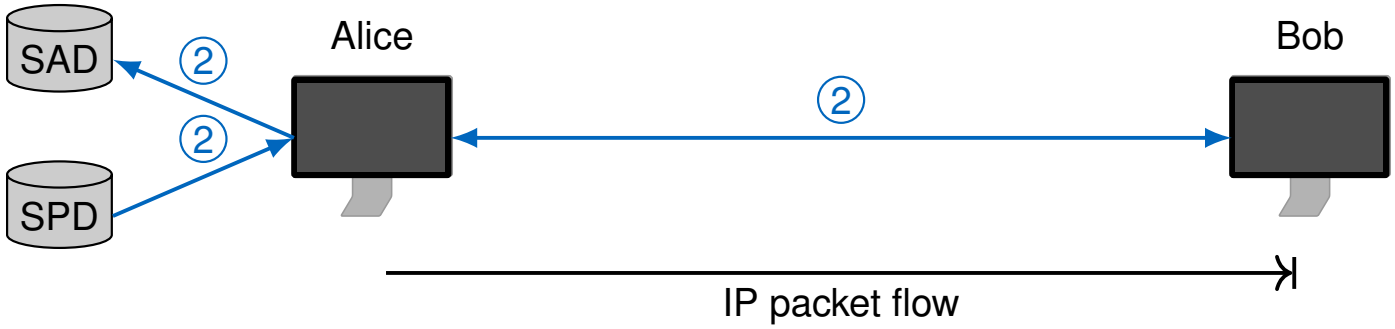


Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. Determine if and how the outgoing packet needs to be secured
   - Perform a lookup in the SPD based on traffic selectors
   - If the policy specifies *discard* then drop the packet $\Rightarrow$ Done
   - If the policy does not need to be secured, send it $\Rightarrow$ Done

Processing of IPSec Traffic

Outgoing Traffic

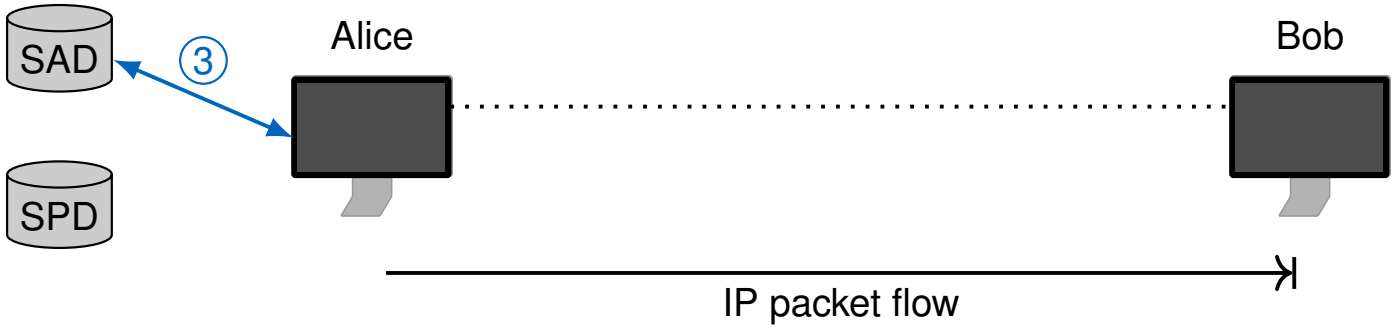


Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. Determine if and how the outgoing packet needs to be secured
   - Perform a lookup in the SPD based on traffic selectors
   - If the policy specifies *discard* then drop the packet $\Rightarrow$ Done
   - If the policy does not need to be secured, send it $\Rightarrow$ Done
2. Determine which SA should be applied to the packet
   - If no SA is established perform IKE
   - There may be more than one SA matching the packet (e.g. one for AH, one for ESP)

Processing of IPSec Traffic

Outgoing Traffic

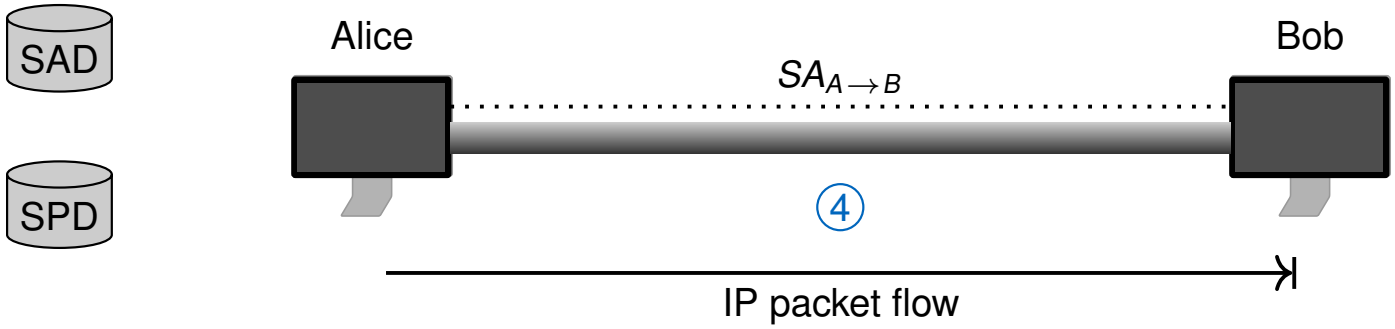


Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. Determine if and how the outgoing packet needs to be secured
   - Perform a lookup in the SPD based on traffic selectors
   - If the policy specifies *discard* then drop the packet $\Rightarrow$ Done
   - If the policy does not need to be secured, send it $\Rightarrow$ Done
2. Determine which SA should be applied to the packet
   - If no SA is established perform IKE
   - There may be more than one SA matching the packet (e.g. one for AH, one for ESP)
3. Look up the determined or freshly created SA in the SAD

Processing of IPSec Traffic

Outgoing Traffic

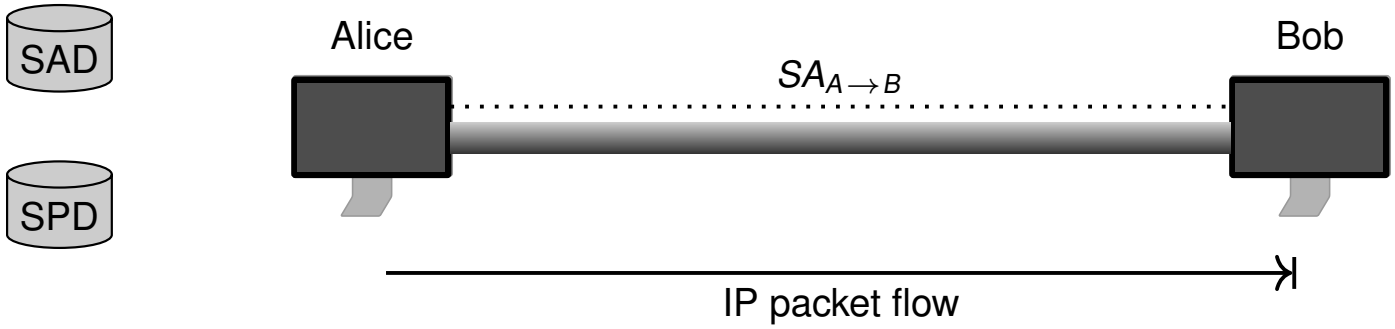


Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. Determine if and how the outgoing packet needs to be secured
   - Perform a lookup in the SPD based on traffic selectors
   - If the policy specifies *discard* then drop the packet $\Rightarrow$ Done
   - If the policy does not need to be secured, send it $\Rightarrow$ Done
2. Determine which SA should be applied to the packet
   - If no SA is established perform IKE
   - There may be more than one SA matching the packet (e.g. one for AH, one for ESP)
3. Look up the determined or freshly created SA in the SAD
4. Perform the *security transforms*, specified in the SA
   - This results in the construction of an AH or ESP header
   - Possibly a new (outer) IP header will be created (tunnel mode)

Processing of IPSec Traffic

Outgoing Traffic

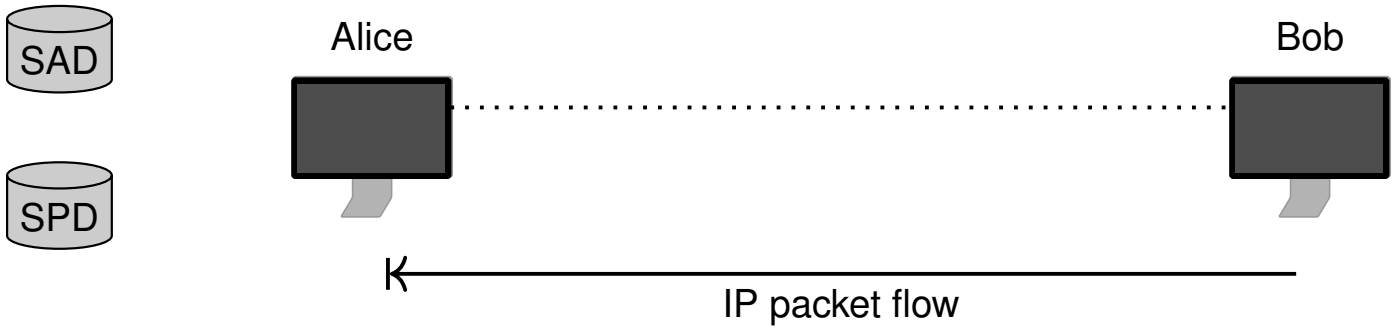


Alice wants to send data to Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. Determine if and how the outgoing packet needs to be secured
   - Perform a lookup in the SPD based on traffic selectors
   - If the policy specifies *discard* then drop the packet $\Rightarrow$ Done
   - If the policy does not need to be secured, send it $\Rightarrow$ Done
2. Determine which SA should be applied to the packet
   - If no SA is established perform IKE
   - There may be more than one SA matching the packet (e.g. one for AH, one for ESP)
3. Look up the determined or freshly created SA in the SAD
4. Perform the *security transforms*, specified in the SA
   - This results in the construction of an AH or ESP header
   - Possibly a new (outer) IP header will be created (tunnel mode)
5. Send the resulting packet $\Rightarrow$ Done

Processing of IPSec Traffic

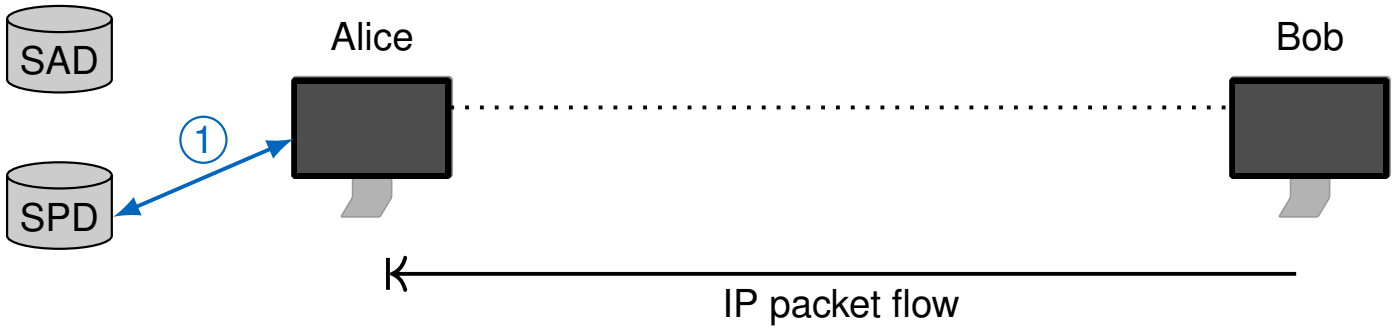
Incoming Traffic



Alice receives data from Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

Processing of IPSec Traffic

Incoming Traffic

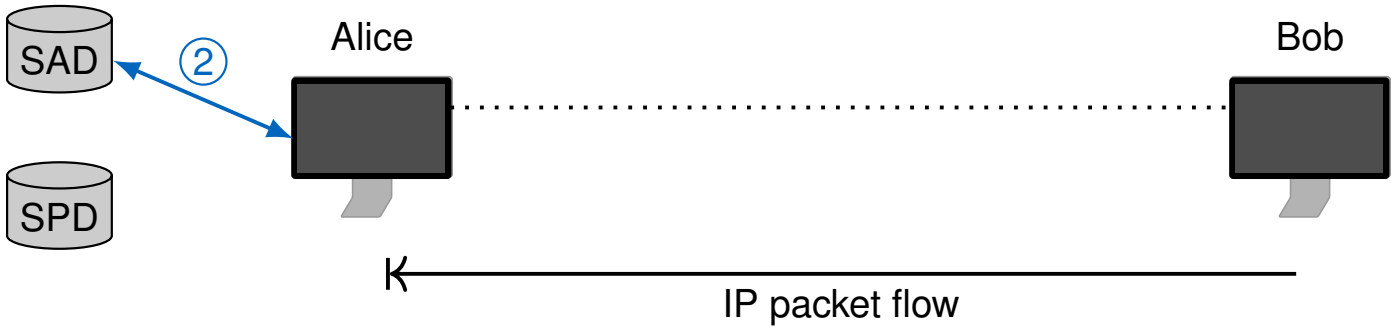


Alice receives data from Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. If packet contains an IPSec header
   - Perform a lookup in the SPD, if Alice is supposed to process the packet
   - Retrieve the respective policy

Processing of IPSec Traffic

Incoming Traffic

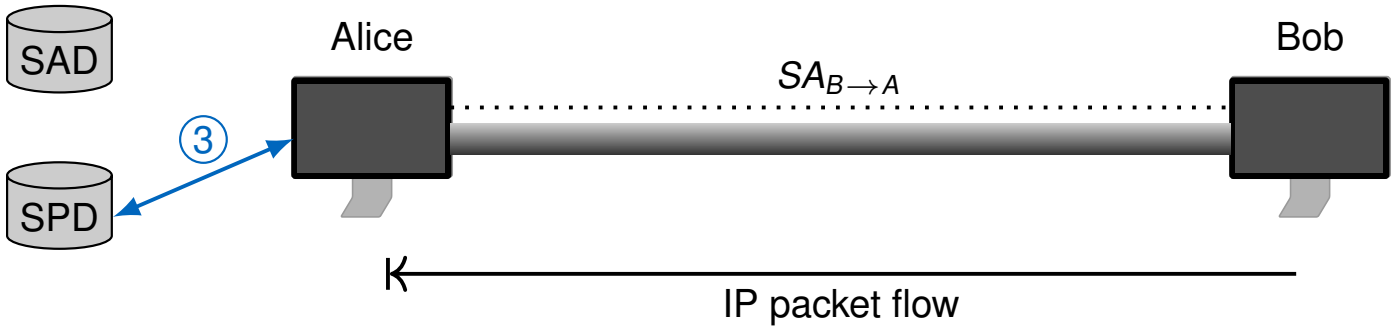


Alice receives data from Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. If packet contains an IPSec header
   - Perform a lookup in the SPD, if Alice is supposed to process the packet
   - Retrieve the respective policy
2. If Alice is supposed to process the packet
   - Extract the SPI from the IPSec header, look up the SA in the SAD and perform the appropriate processing
   - If there's no SA referenced by the SPI $\Rightarrow$ Drop the packet

Processing of IPSec Traffic

Incoming Traffic

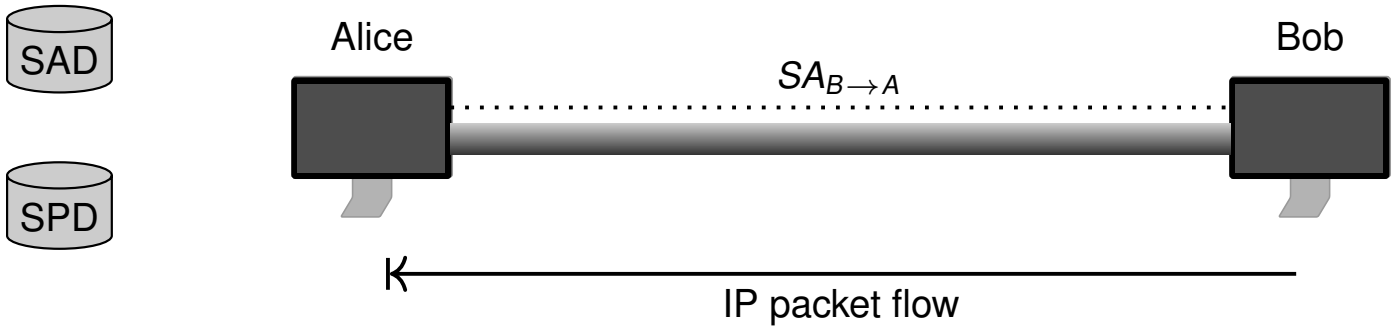


Alice receives data from Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. If packet contains an IPSec header
   - Perform a lookup in the SPD, if Alice is supposed to process the packet
   - Retrieve the respective policy
2. If Alice is supposed to process the packet
   - Extract the SPI from the IPSec header, look up the SA in the SAD and perform the appropriate processing
   - If there's no SA referenced by the SPI $\Rightarrow$ Drop the packet
3. Determine if and how the packet should have been protected
   - Perform a lookup in the SPD, evaluating the inner IP header in case of tunneled packets
   - If the respective policy specifies *discard* $\Rightarrow$ Drop the packet
   - If the protection of the packet did not match the policy $\Rightarrow$ Drop the packet

Processing of IPSec Traffic

Incoming Traffic

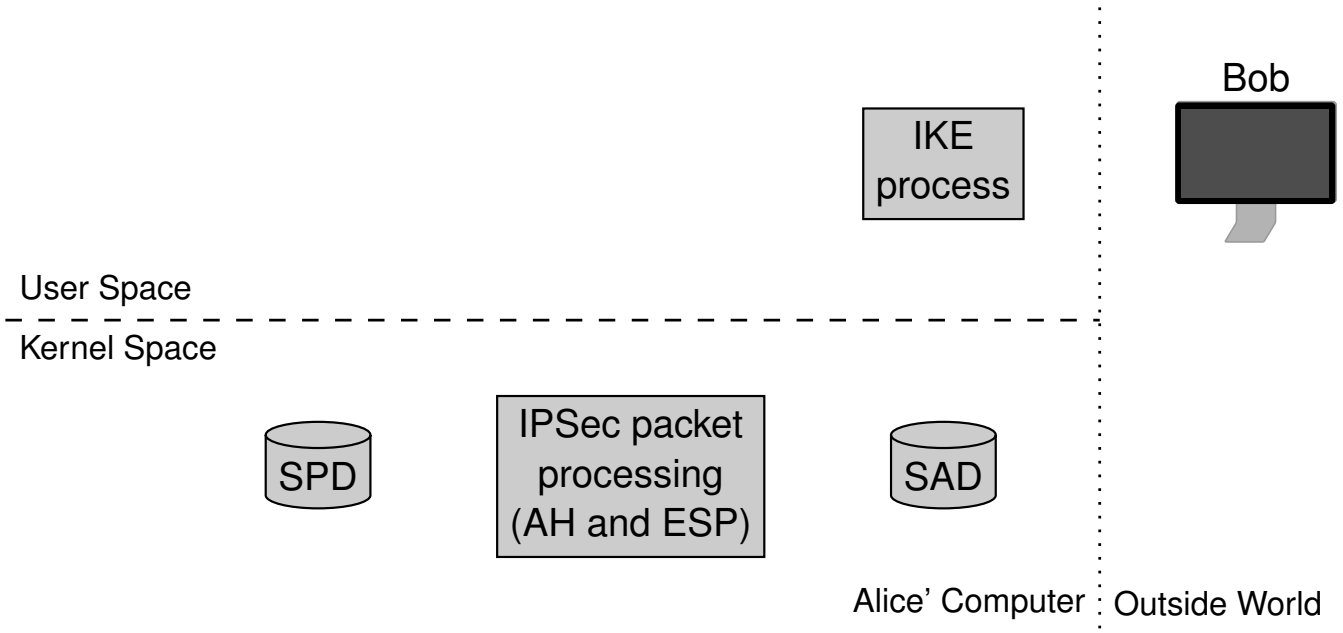


Alice receives data from Bob. To support IPSec, the IP layer of Alice has to perform the following steps:

1. If packet contains an IPSec header
   - Perform a lookup in the SPD, if Alice is supposed to process the packet
   - Retrieve the respective policy
2. If Alice is supposed to process the packet
   - Extract the SPI from the IPSec header, look up the SA in the SAD and perform the appropriate processing
   - If there's no SA referenced by the SPI $\Rightarrow$ Drop the packet
3. Determine if and how the packet should have been protected
   - Perform a lookup in the SPD, evaluating the inner IP header in case of tunneled packets
   - If the respective policy specifies *discard* $\Rightarrow$ Drop the packet
   - If the protection of the packet did not match the policy $\Rightarrow$ Drop the packet
4. Deliver to the appropriate protocol entity (e.g. network / transport layer)

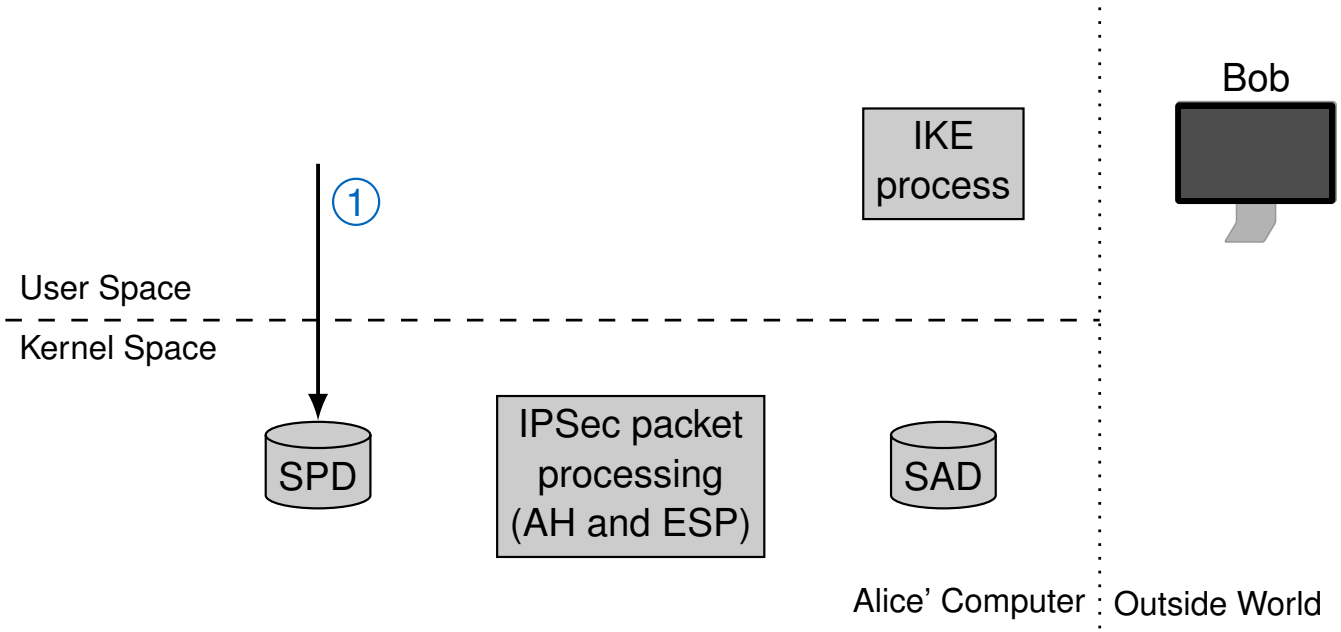
Processing of IPSec Traffic

Architecture View



Processing of IPSec Traffic

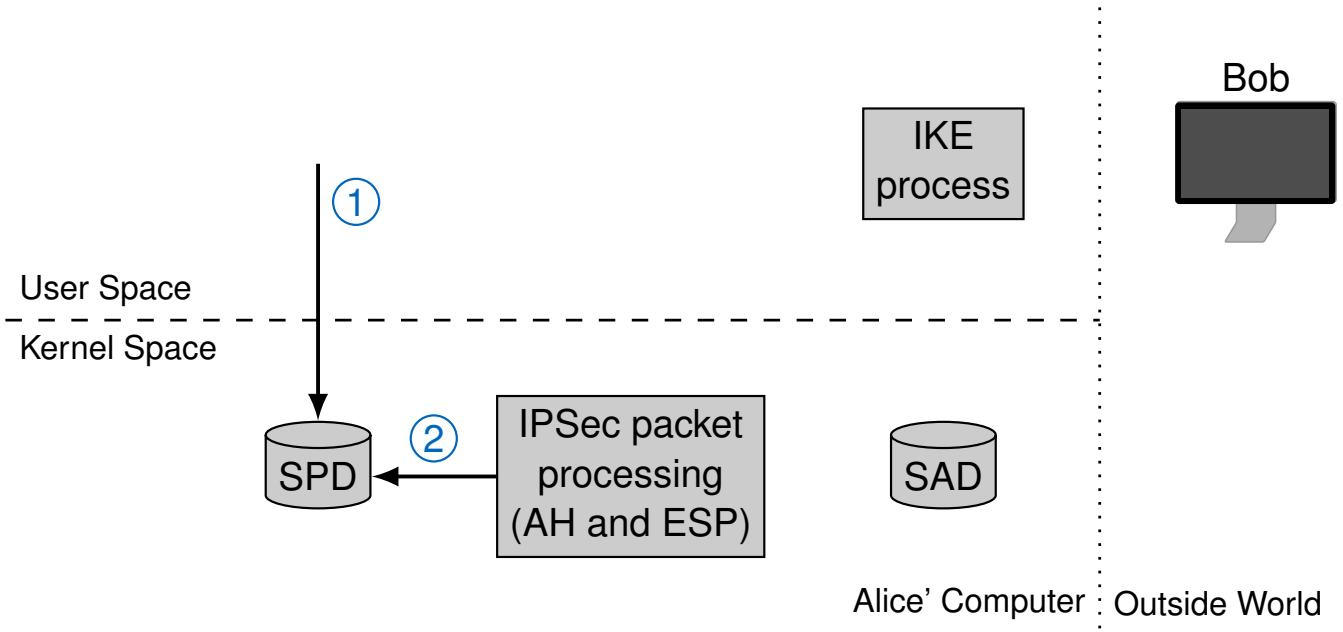
Architecture View



1 The administrator sets a policy in SPD

Processing of IPSec Traffic

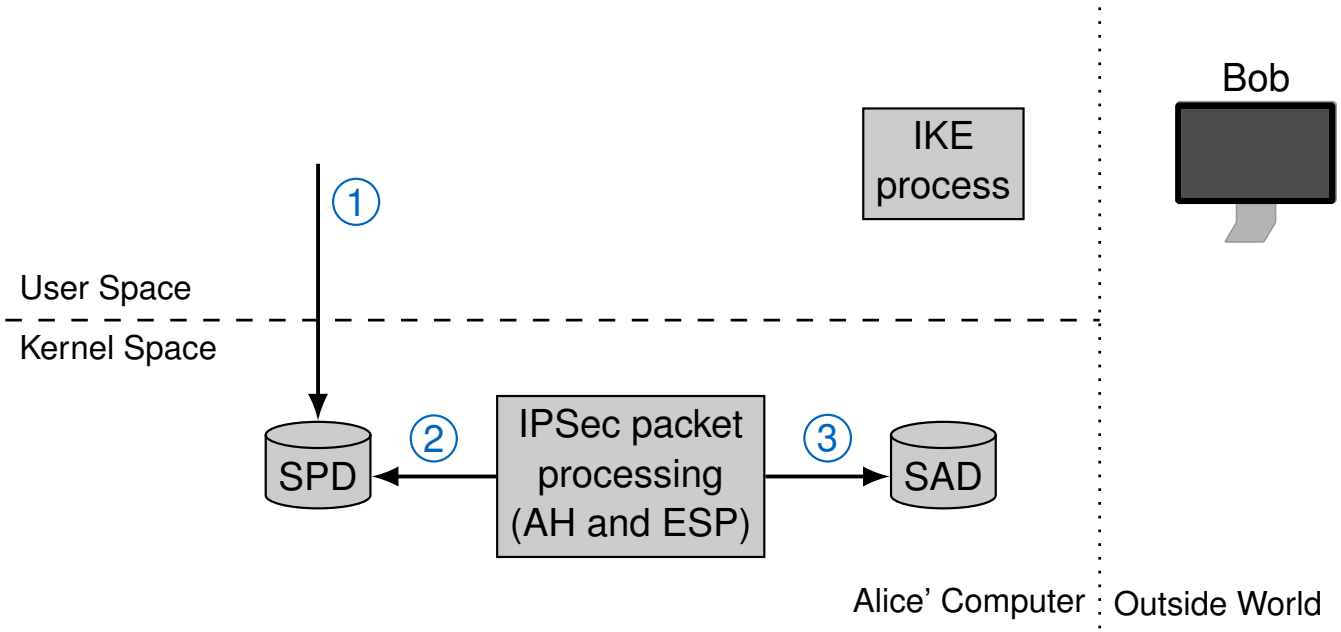
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet

Processing of IPSec Traffic

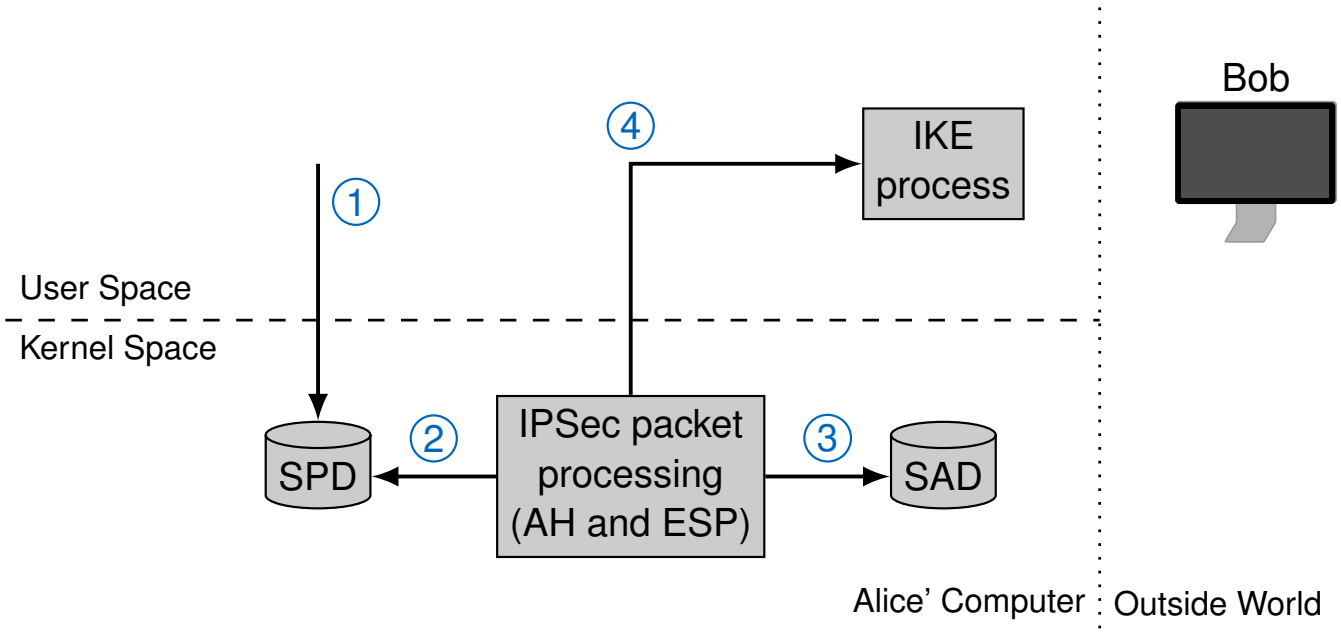
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet
3. If IPSec is required, then the IPSec module looks for the IPSec SA in the SAD

Processing of IPSec Traffic

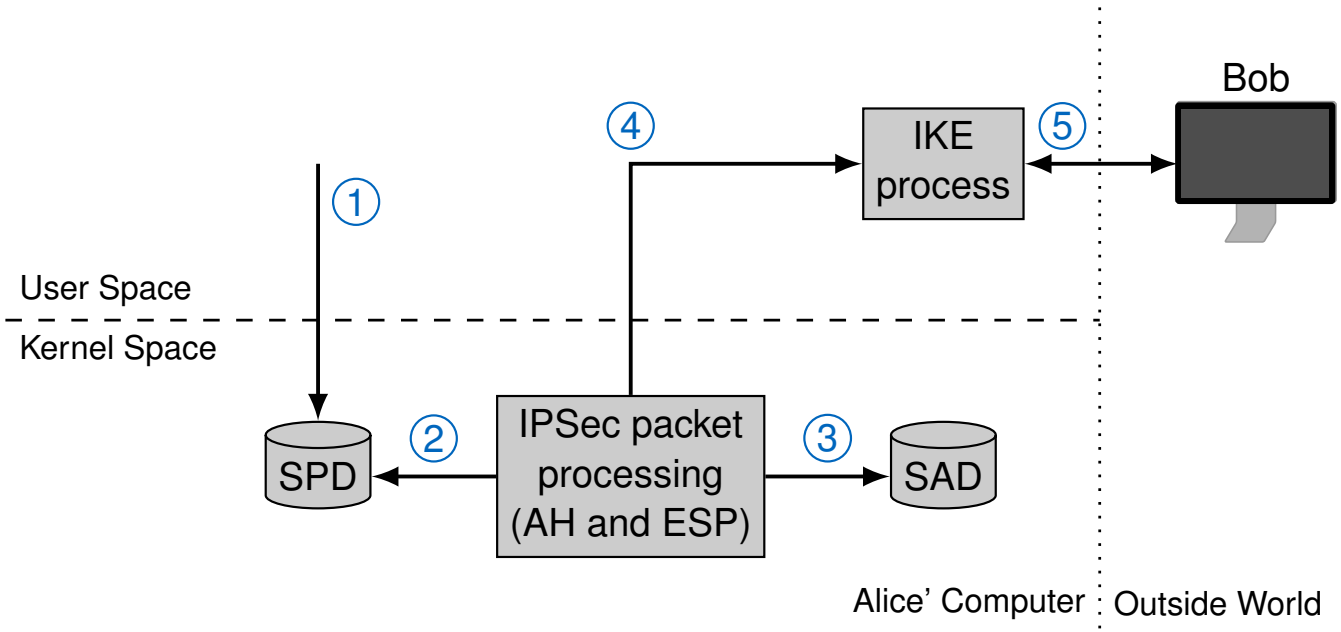
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet
3. If IPSec is required, then the IPSec module looks for the IPSec SA in the SAD
4. If there is no SA yet, the IPSec module sends a request to the IKE process to create an SA

Processing of IPSec Traffic

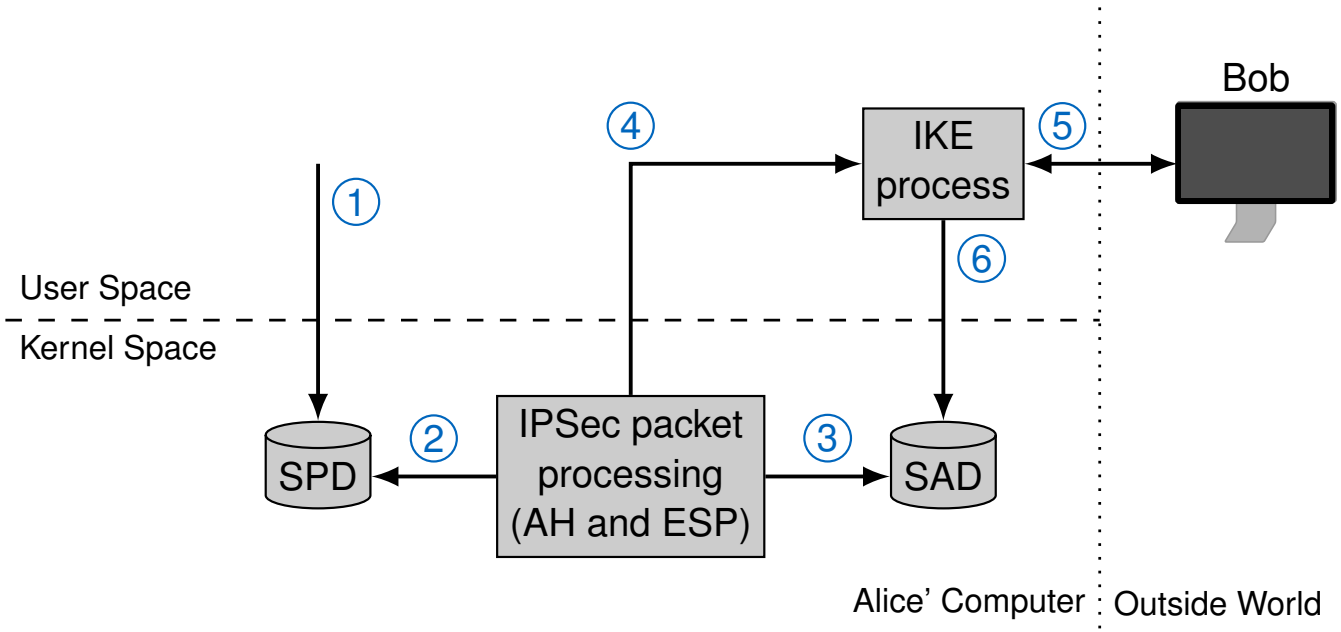
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet
3. If IPSec is required, then the IPSec module looks for the IPSec SA in the SAD
4. If there is no SA yet, the IPSec module sends a request to the IKE process to create an SA
5. The IKE process negotiates keys and crypto algorithms with the peer host using the IKE/IKEv2 protocol

Processing of IPSec Traffic

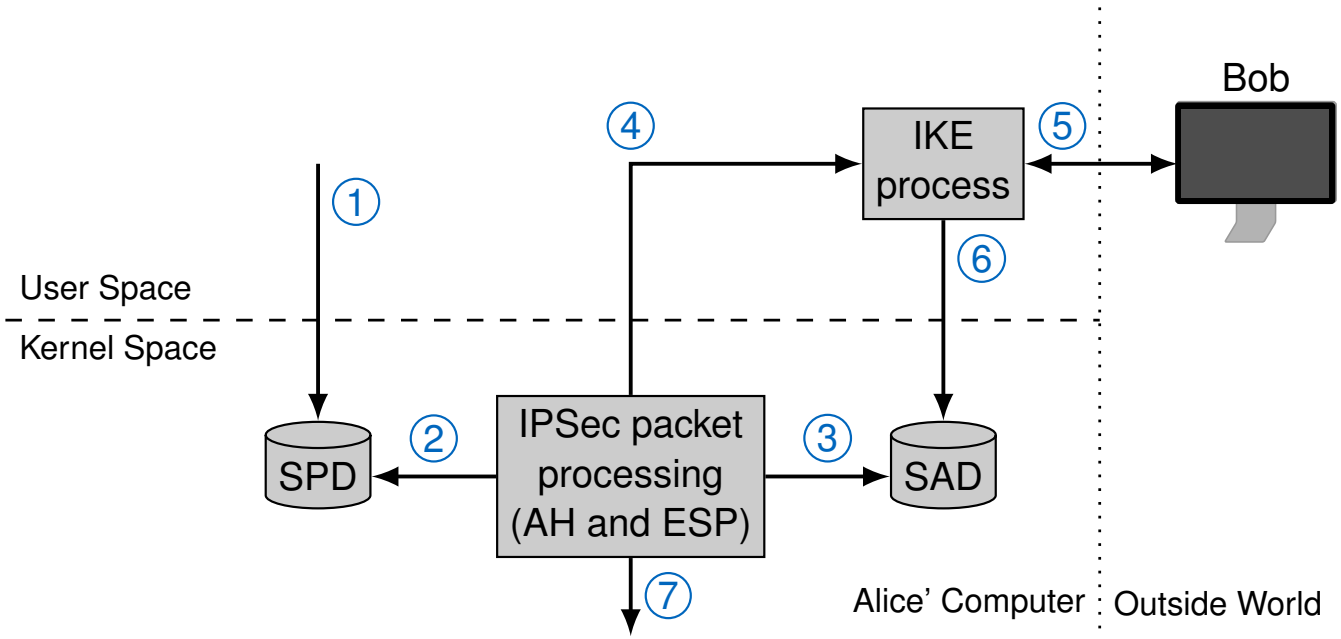
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet
3. If IPSec is required, then the IPSec module looks for the IPSec SA in the SAD
4. If there is no SA yet, the IPSec module sends a request to the IKE process to create an SA
5. The IKE process negotiates keys and crypto algorithms with the peer host using the IKE/IKEv2 protocol
6. The IKE process writes the key and all required parameters into the SAD

Processing of IPSec Traffic

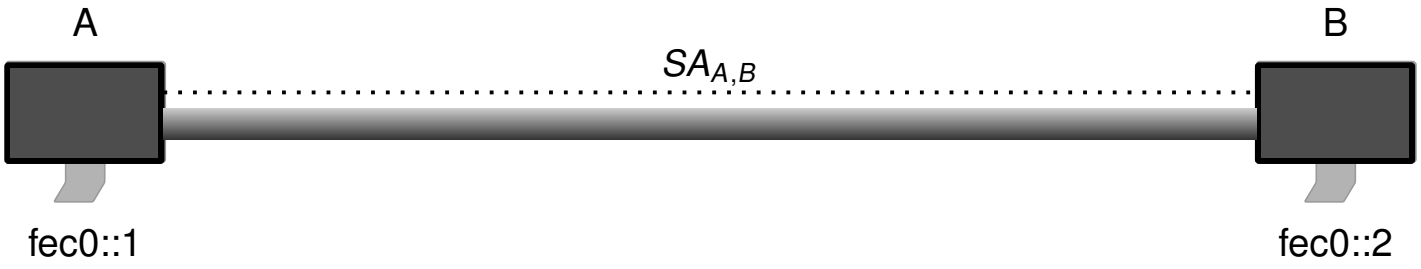
Architecture View



1. The administrator sets a policy in SPD
2. The IPSec processing module refers to the SPD in order to make a decision on applying IPSec on packet
3. If IPSec is required, then the IPSec module looks for the IPSec SA in the SAD
4. If there is no SA yet, the IPSec module sends a request to the IKE process to create an SA
5. The IKE process negotiates keys and crypto algorithms with the peer host using the IKE/IKEv2 protocol
6. The IKE process writes the key and all required parameters into the SAD
7. The IPSec module can now send a packet with applied IPSec

Setup of IPSec Security Policies

Example:
IPv6 connection with *ESP* and *Transport Mode*



Configuration at Host A

- `spdadd fec0::1 fec0::2 any -P out ipsec esp/transport//require;`
- `spdadd fec0::2 fec0::1 any -P in ipsec esp/transport//require;`

where

- First IP corresponds to source in the IP header
- Second IP corresponds to destination in the IP header
- `out` defines the policy to hold for outgoing packets
- `in` defines the policy to hold for incoming packets

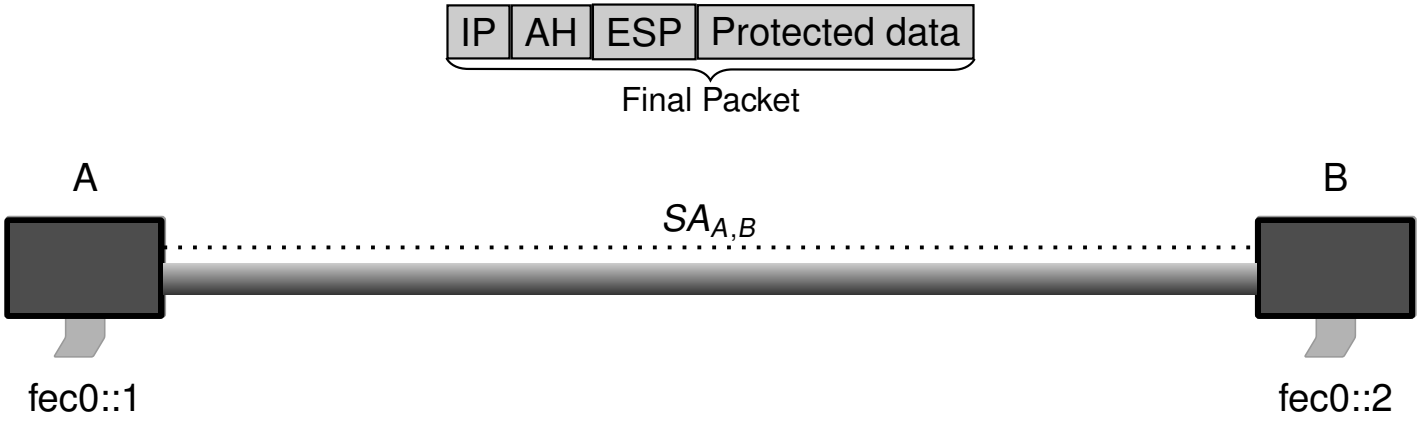
$\Downarrow$

Configuration at Host B:

- `spdadd fec0::2 fec0::1 any -P out ipsec esp/transport//require;`
- `spdadd fec0::1 fec0::2 any -P in ipsec esp/transport//require;`

Setup of IPSec Security Policies

Example:
IPv6 connection with *ESP/Transport* applied first and *AH/Transport* applied next:



Configuration at Host A

- `spdadd fec0::1 fec0::2 any -P out ipsec esp/transport//require ah/transport//require;`
- `spdadd fec0::2 fec0::1 any -P in ipsec esp/transport//require ah/transport//require;`

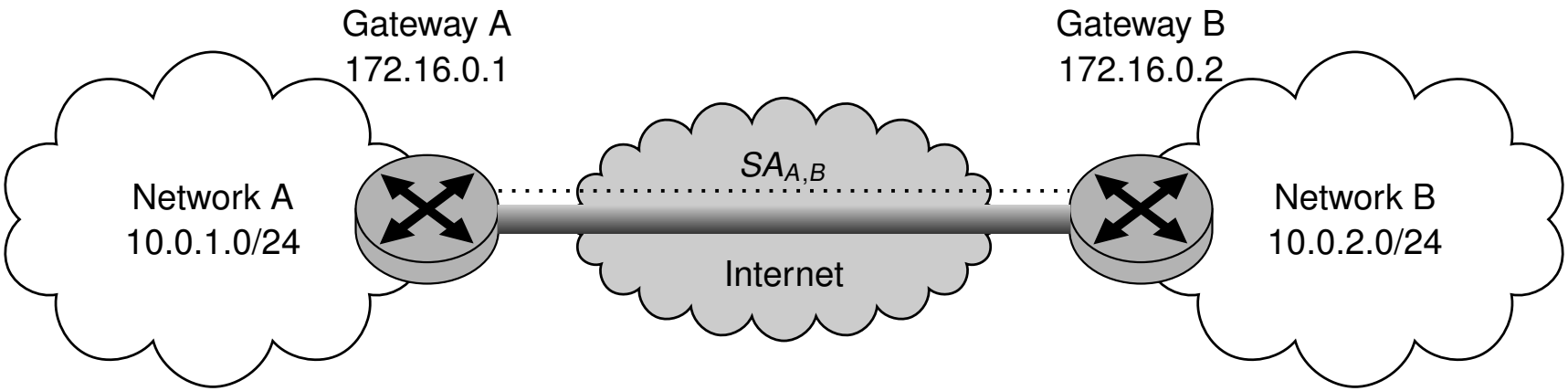
$\Downarrow$

Configuration at Host B:

- `spdadd fec0::2 fec0::1 any -P out ipsec esp/transport//require ah/transport//require;`
- `spdadd fec0::1 fec0::2 any -P in ipsec esp/transport//require ah/transport//require;`

Setup of IPSec Security Policies

Example:
ESP Tunnel for VPN



Configuration at Gateway A

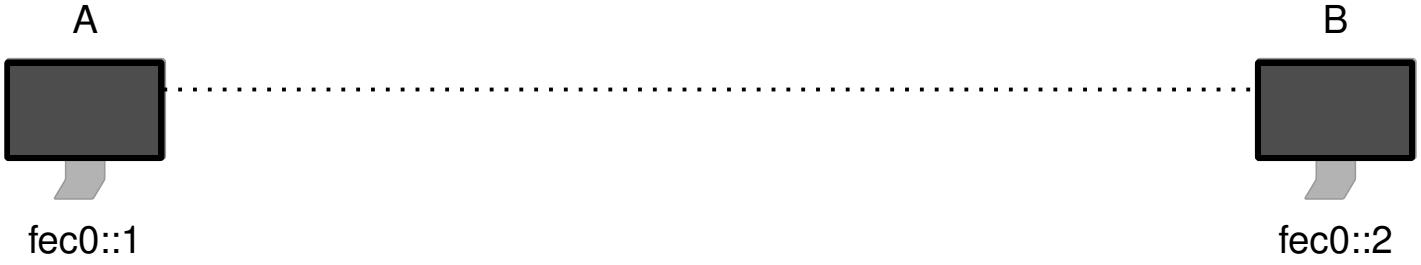
- `spdadd 10.0.1.0/24 10.0.2.0/24 any -P out ipsec esp/tunnel/172.16.0.1-172.16.0.2/require;`
- `spdadd 10.0.2.0/24 10.0.1.0/24 any -P in ipsec esp/tunnel/172.16.0.2-172.16.0.1/require;`

$\Downarrow$

Configuration at Gateway B:

- `spdadd 10.0.2.0/24 10.0.1.0/24 any -P out ipsec esp/tunnel/172.16.0.2-172.16.0.1/require;`
- `spdadd 10.0.1.0/24 10.0.2.0/24 any -P in ipsec esp/tunnel/172.16.0.1-172.16.0.2/require;`

Setup of IPSec Security Associations



Example:
Manually setting up an *AH SA*:

```
# add src dst proto spi -A authalgo key;
add fec0::1 fec0::2 ah 700 -A hmac-md5 0xbf9a081e7ebdd4fa824c822ed94f5226;
add fec0::2 fec0::1 ah 800 -A hmac-md5 0xbf9a081e7ebdd4fa824c822ed94f5226;
```

Manually setting up an *ESP SA*:

```
# add src dst proto spi -E encalgo key;
add fec0::1 fec0::2 esp 701 -E 3des-cbc 0xdafb418410b2ca6a2ba144561fab354640080e5b7a;
add fec0::2 fec0::1 esp 801 -E 3des-cbc 0xdafb418410b2ca6a2ba144561fab354640080e5b7a;
```

WARNING: Setting up an SA manually is error prone!

- The administrator might choose insecure keys
- The set of SAs might be inconsistent
- It is better to rely on an IKE daemon for setting up SAs

Chapter 12: IPSec

Introduction

IPSec Architecture

IPSec Security Protocols

- Encapsulating Security Payload (ESP)
- Authentication Header (AH)
- Cryptographic Algorithms in IPSec

Entity Authentication and Key Exchange

WireGuard

Encapsulating Security Payload (ESP)

Recap:

ESP is a generic security protocol that provides replay protection and one or both of

- Confidentiality
  - Encryption of encapsulated packets
  - Encryption of packet payloads
- Data origin authentication
  - Creating and adding MAC to packets

The ESP definition is divided into two parts:

[RFC4303] Definition of the base protocol

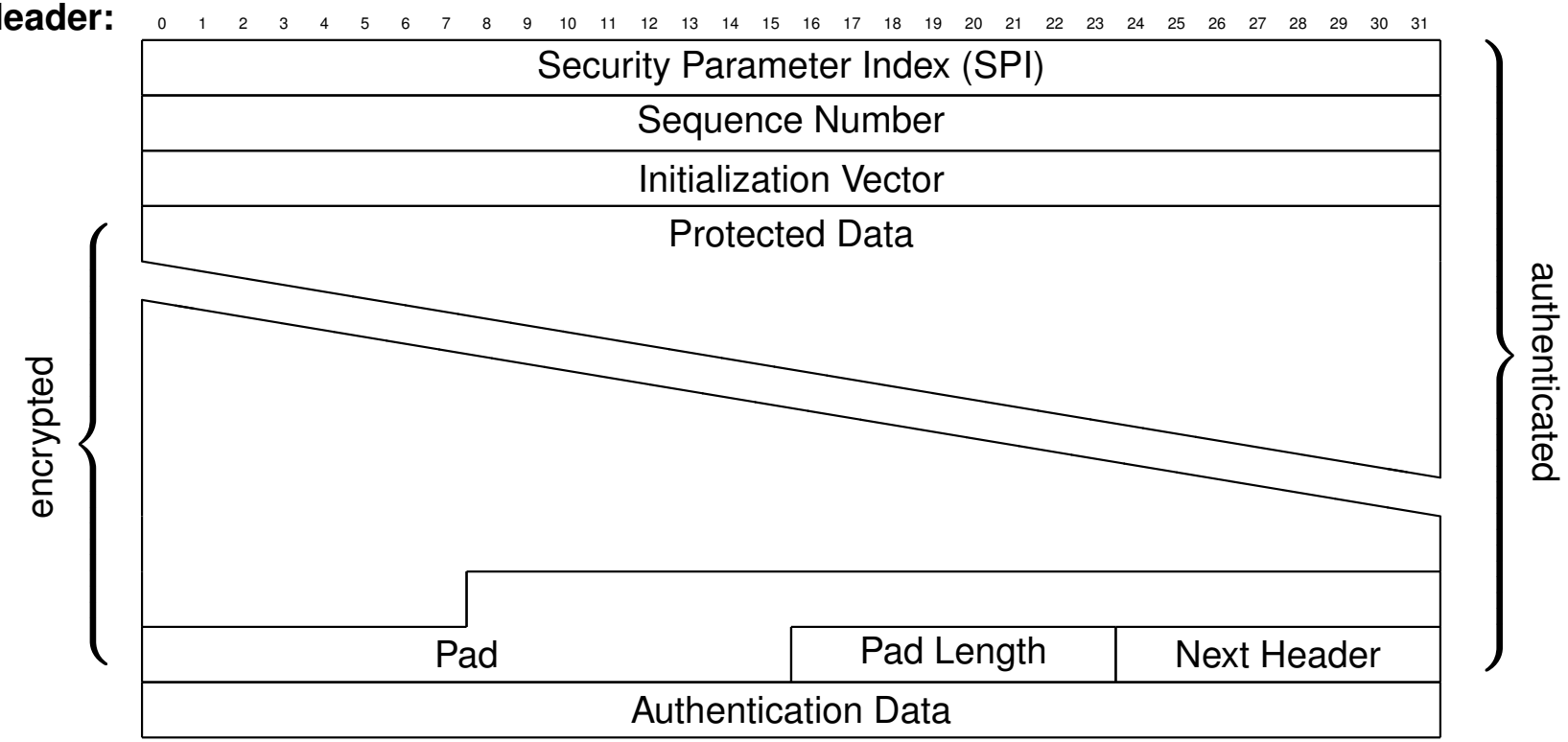
- Definition of header and trailer format
- Basic protocol processing
- Tunnel and transport mode operation

[RFC4305] Implementation requirements for AH/ESP

- Suitable authentication algorithms and modes
- Suitable encryption algorithms and modes
- Security Considerations

Encapsulating Security Payload (ESP)

ESP Header:

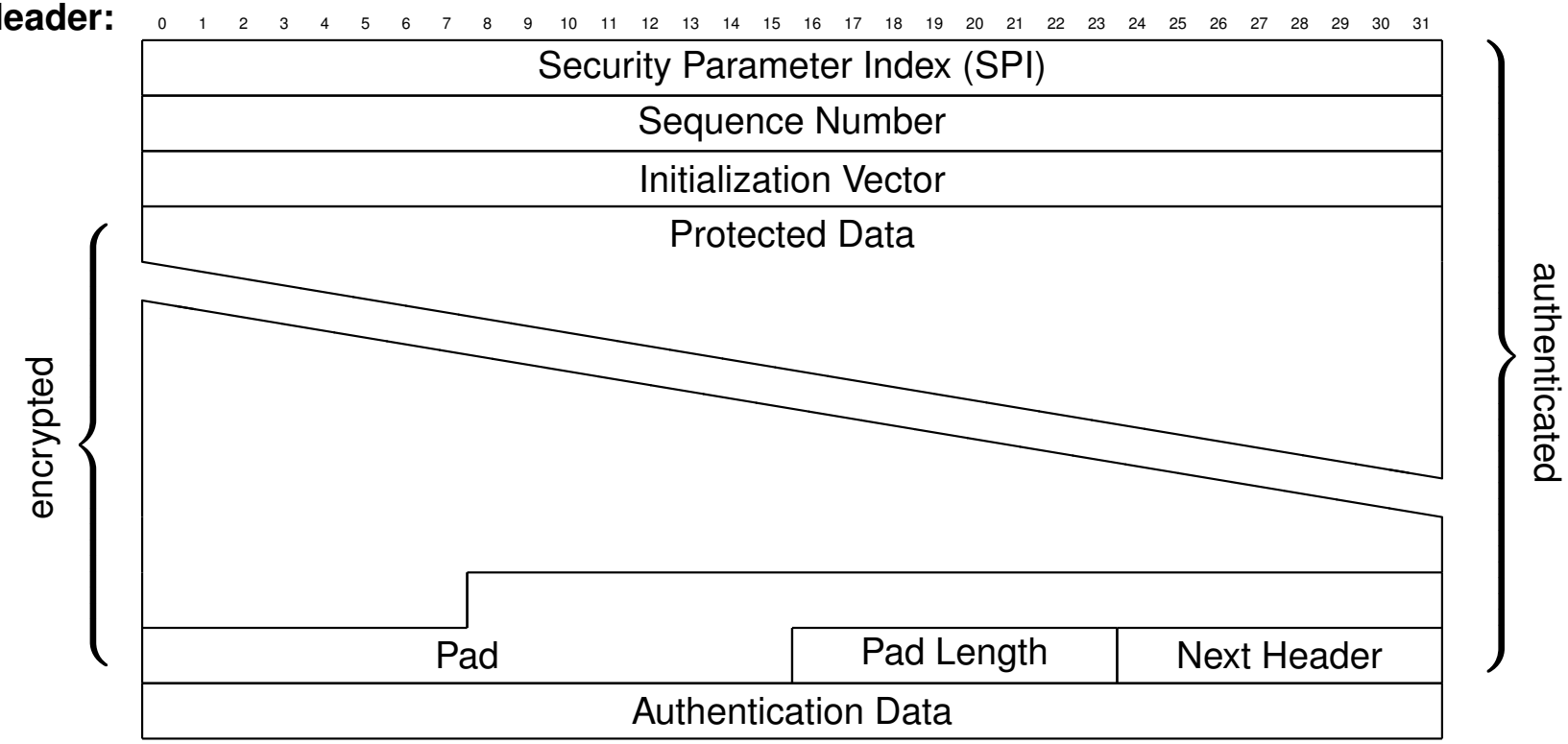


The ESP immediately follows an IP/AH header and is indicated by `Next Header = 50`

- SPI
  - Indicates the SA to be used
  - Determined by the receiving side
- Sequence Number
- Initialization Vector
  - Set if required by crypto algorithm
  - Transmitted as cleartext in every packet
- Pad
  - Padding up to required block length (crypto)
  - Padding to right-justify `Pad Length` and `NH`
  - Need to be in high-order 16bit of 32bit word
- Pad Length

Encapsulating Security Payload (ESP)

ESP Header:

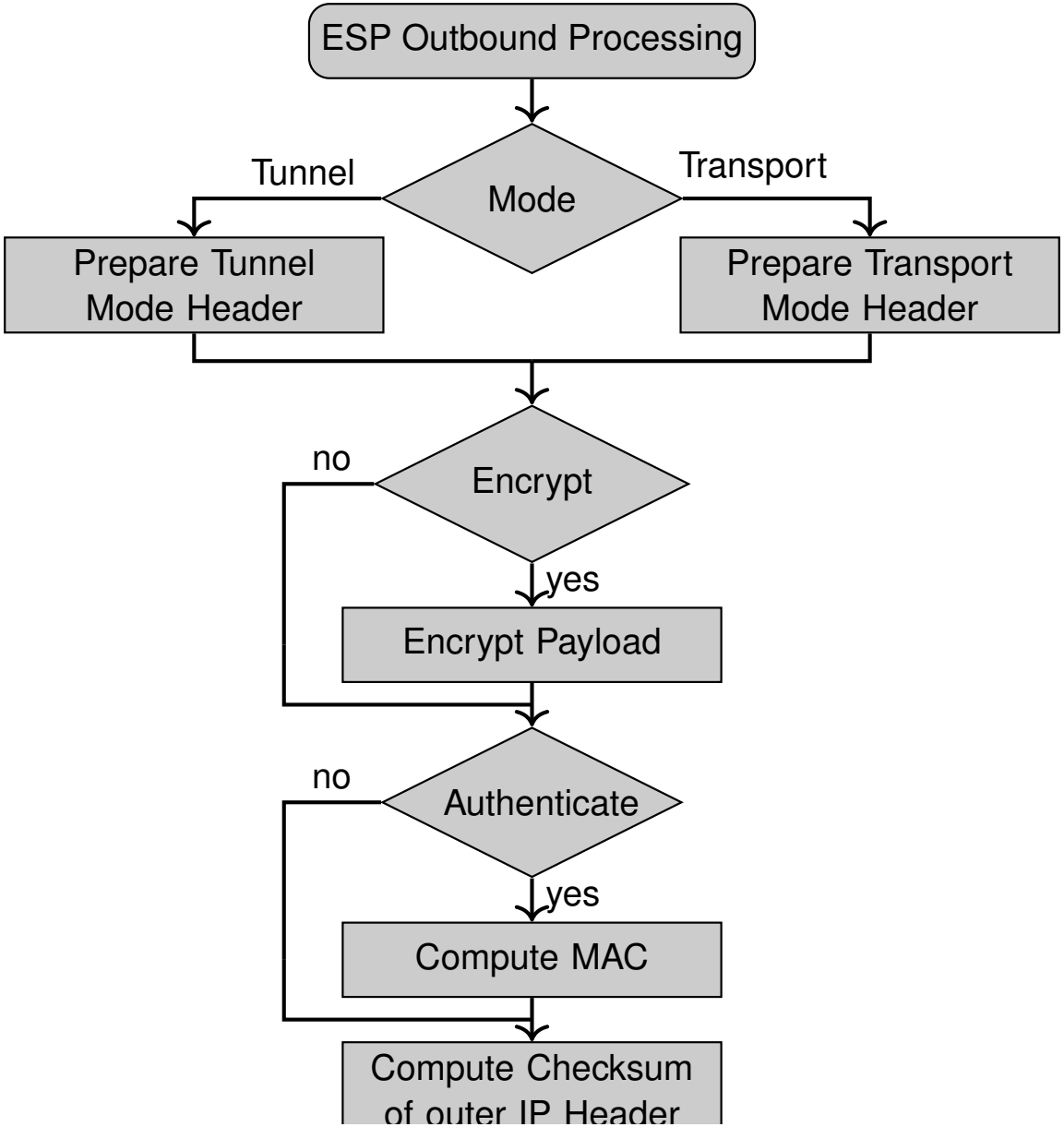


The ESP immediately follows an IP/AH header and is indicated by `Next Header = 50`

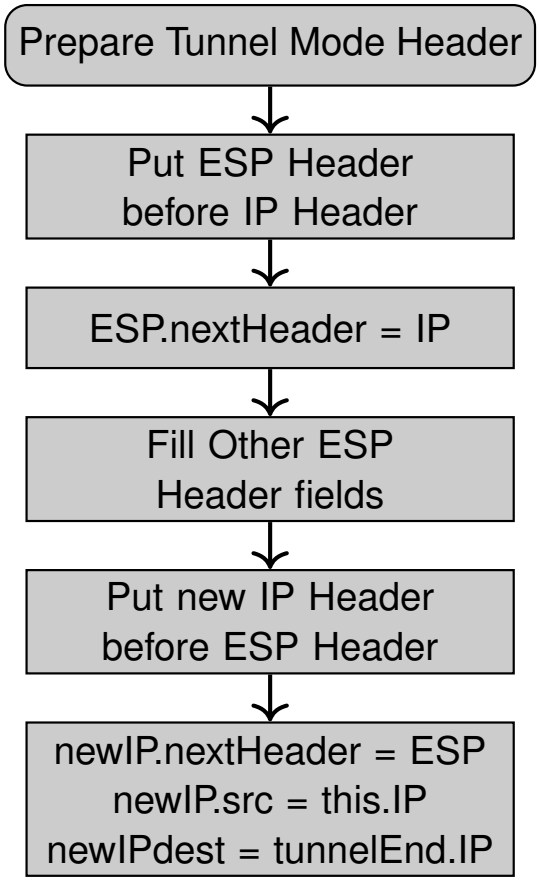
- Next Header
  - Indicates the encapsulated payload
  - In case of tunnel mode ⇒ IP
  - In case of transport mode ⇒ TCP, UDP, ...
- Authentication Data
  - Field is optional
  - Contains a MAC if present

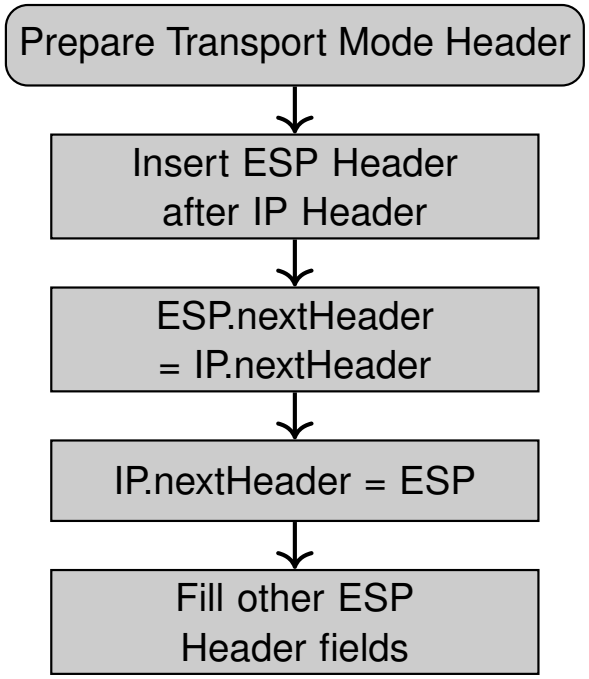
Encapsulating Security Payload (ESP)

Outbound Processing

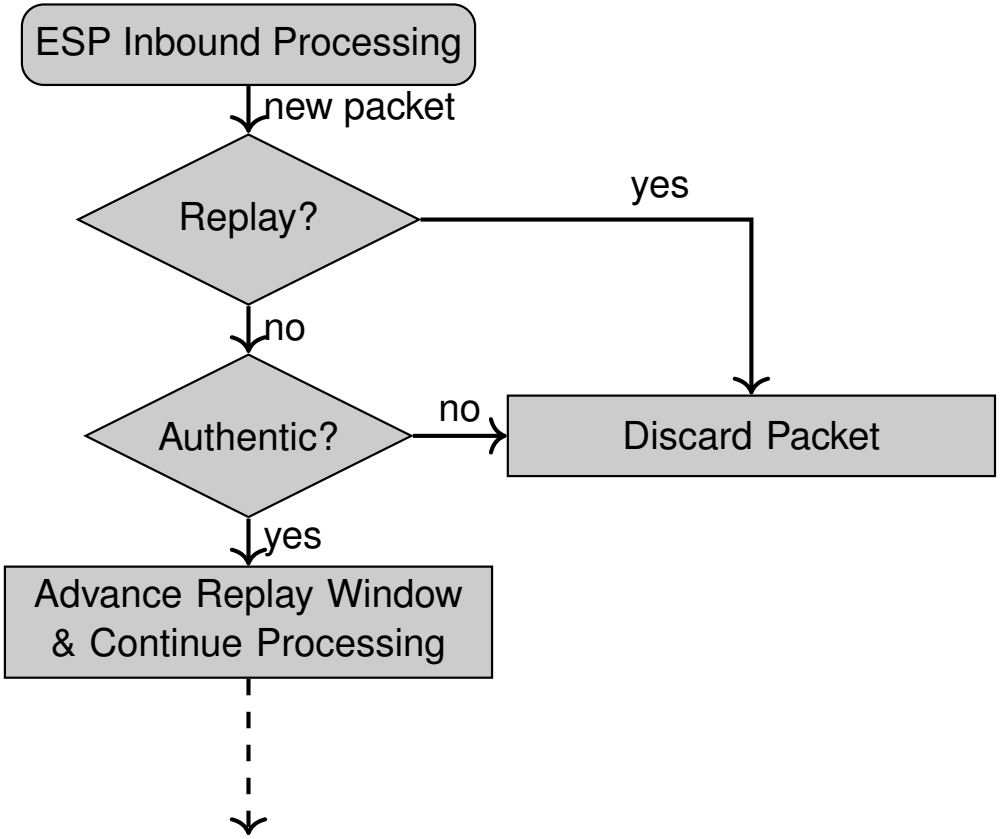


Outbound Processing

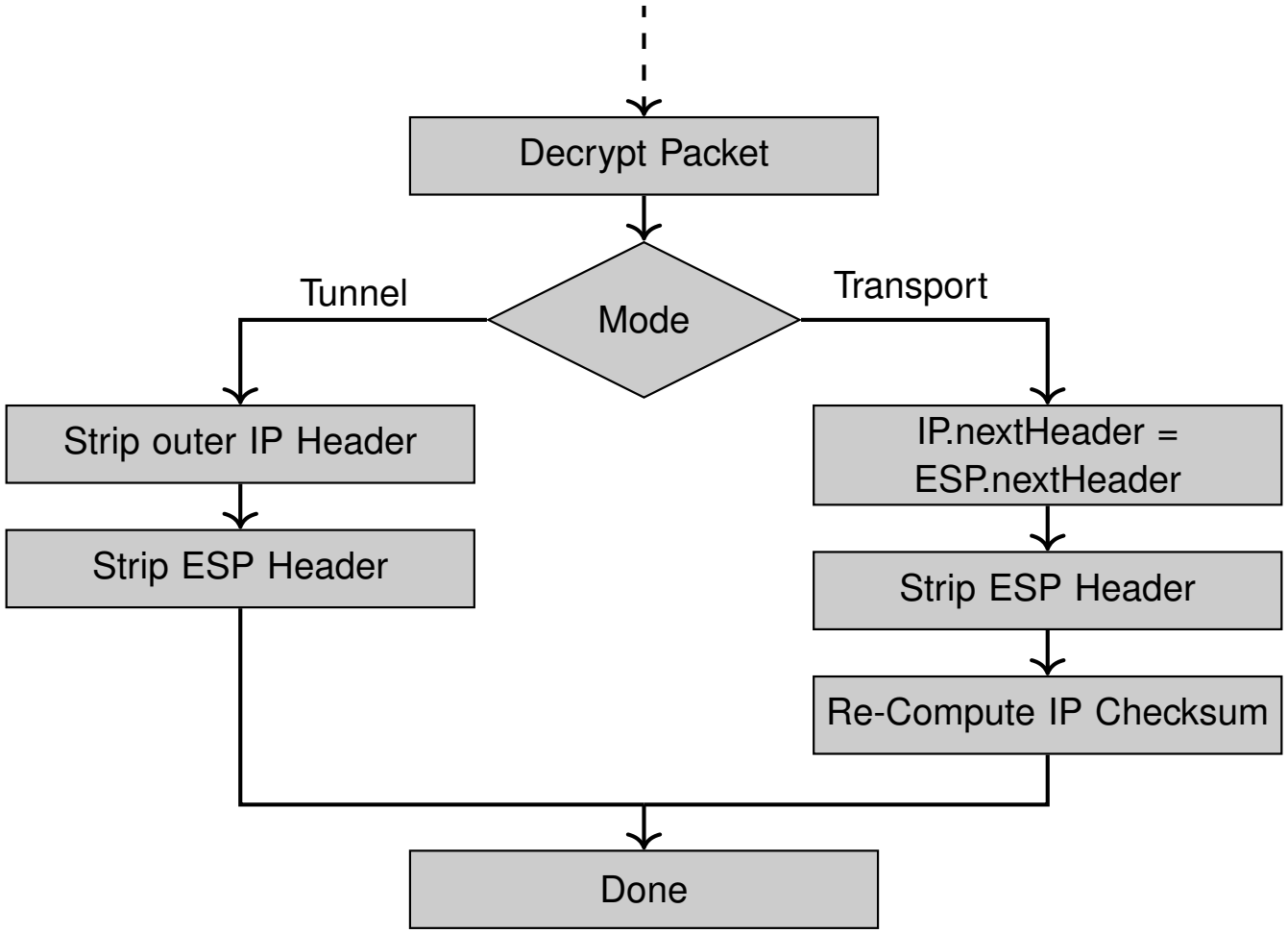




Inbound Processing



Inbound Processing



Authentication Header (AH)

Recap:

AH is a generic security protocol that provides

- Replay protection
- Data origin authentication
  - Creating and adding MAC to packets

Like the ESP, the AH definition is divided into two parts:

[RFC4302] Definition of the base protocol

- Definition of header format
- Basic protocol processing
- Tunnel and transport mode operation

[RFC4305] Implementation requirements for AH/ESP

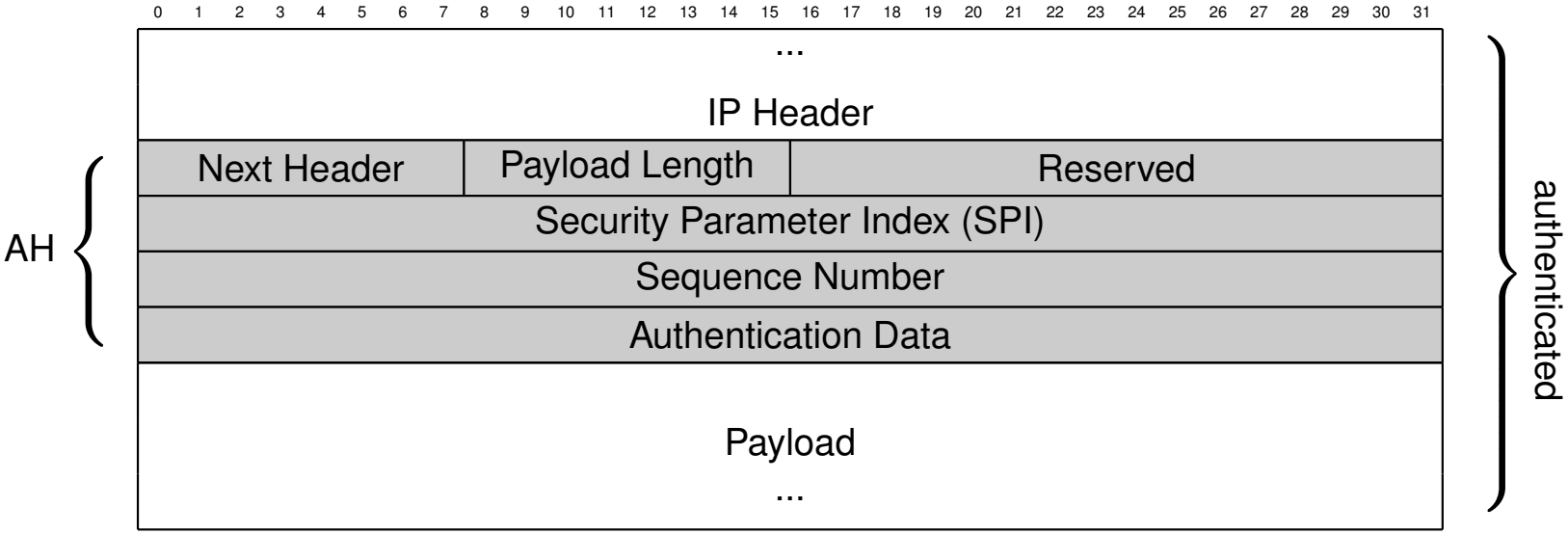
- Usage of algorithms with AH
- HMAC-SHA1-96 [RC2404]
- AES-XCBC-MAC-96 [RFC3566]

Note:

Both ESP and AH can be applied at the same time with different ordering

- If ESP is applied first, AH is outer header
- Advantage: ESP is also protected by AH
- Consequence: Two SAs (one for each of AH/ESP) are needed for each direction

Authentication Header (AH)



The AH immediately follows an IP Header and is indicated by `Next Header = 51`

Note:

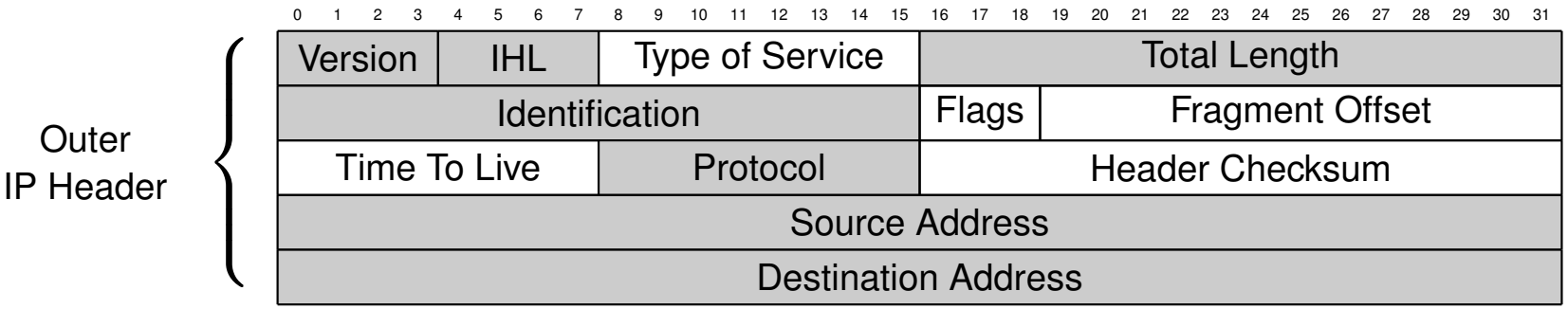
- In tunnel mode the payload is a complete IP packet
- The AH also protects the outer IP Header
- The IP Header contains variable fields
- Wait.

How can we protect the integrity of data, changing in transit?

Authentication Header (AH)

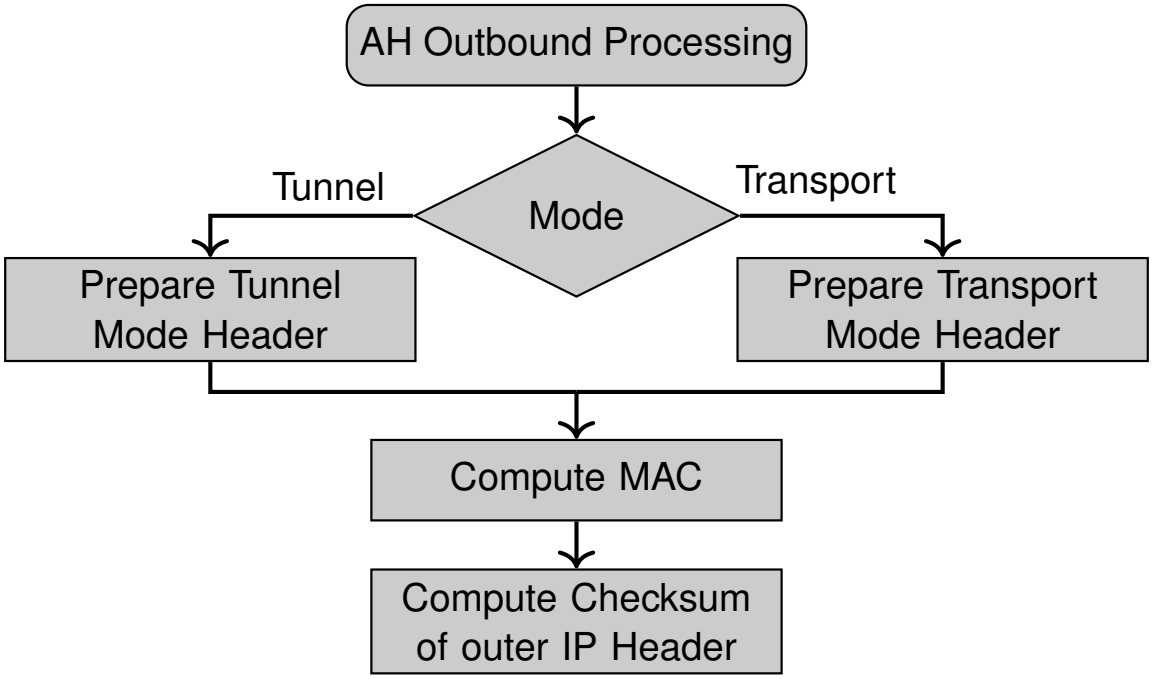
Solution:
Although the AH protects the outer IP Header, some of its fields must not be protected, as they are subject to change during transit.

- This also applies to mutable IPv4 options or IPv6 extensions
- Such fields are assumed to be zero for MAC computation



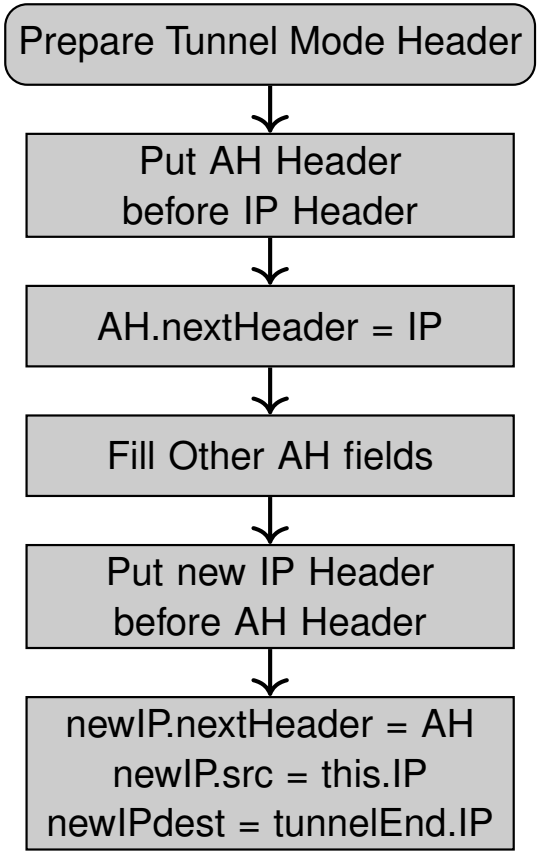
All immutable fields, options and extensions (gray) are protected

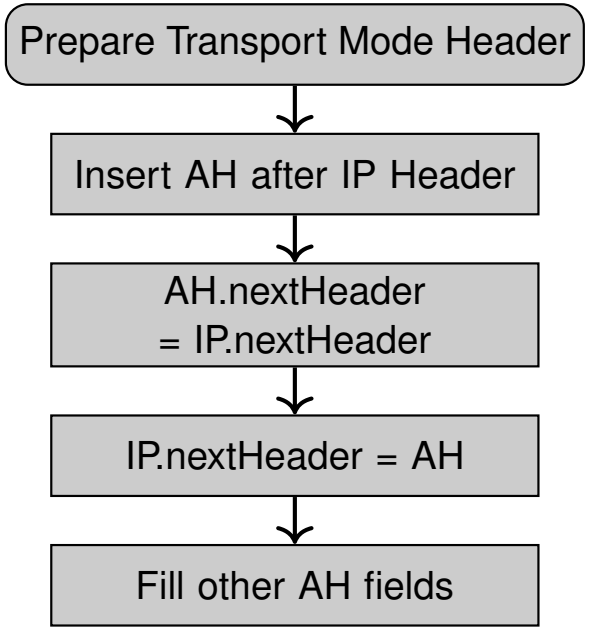
Outbound Processing



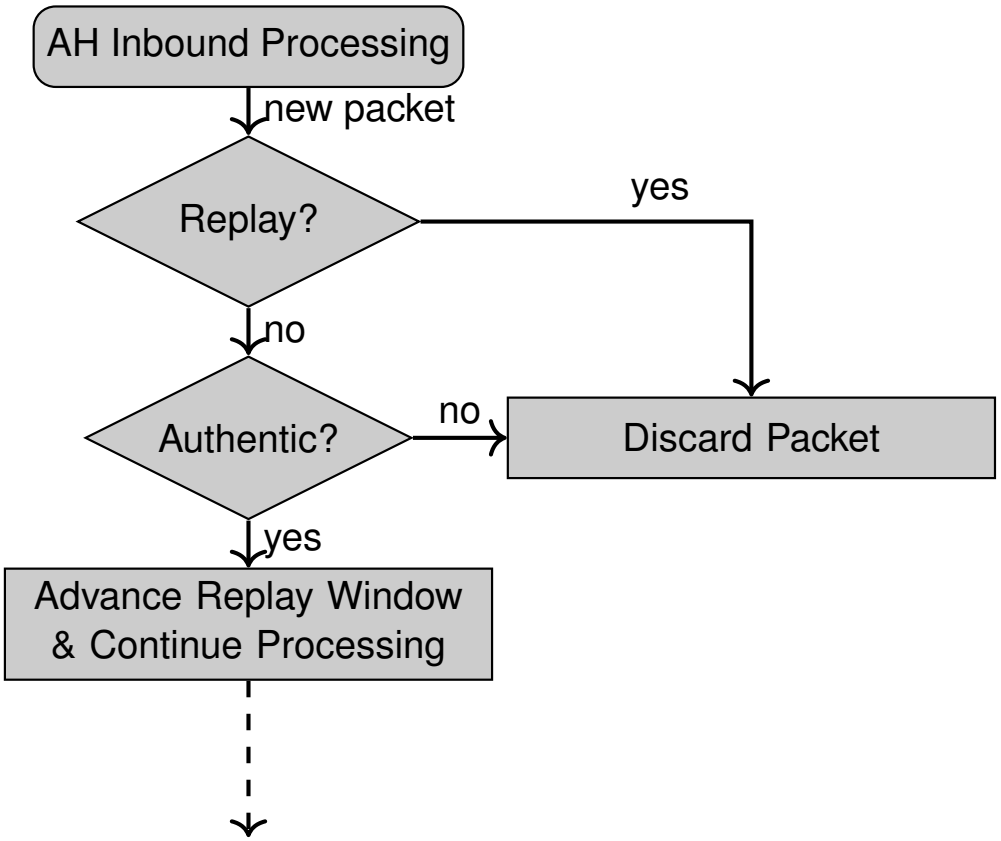
Authentication Header (AH)

Outbound Processing

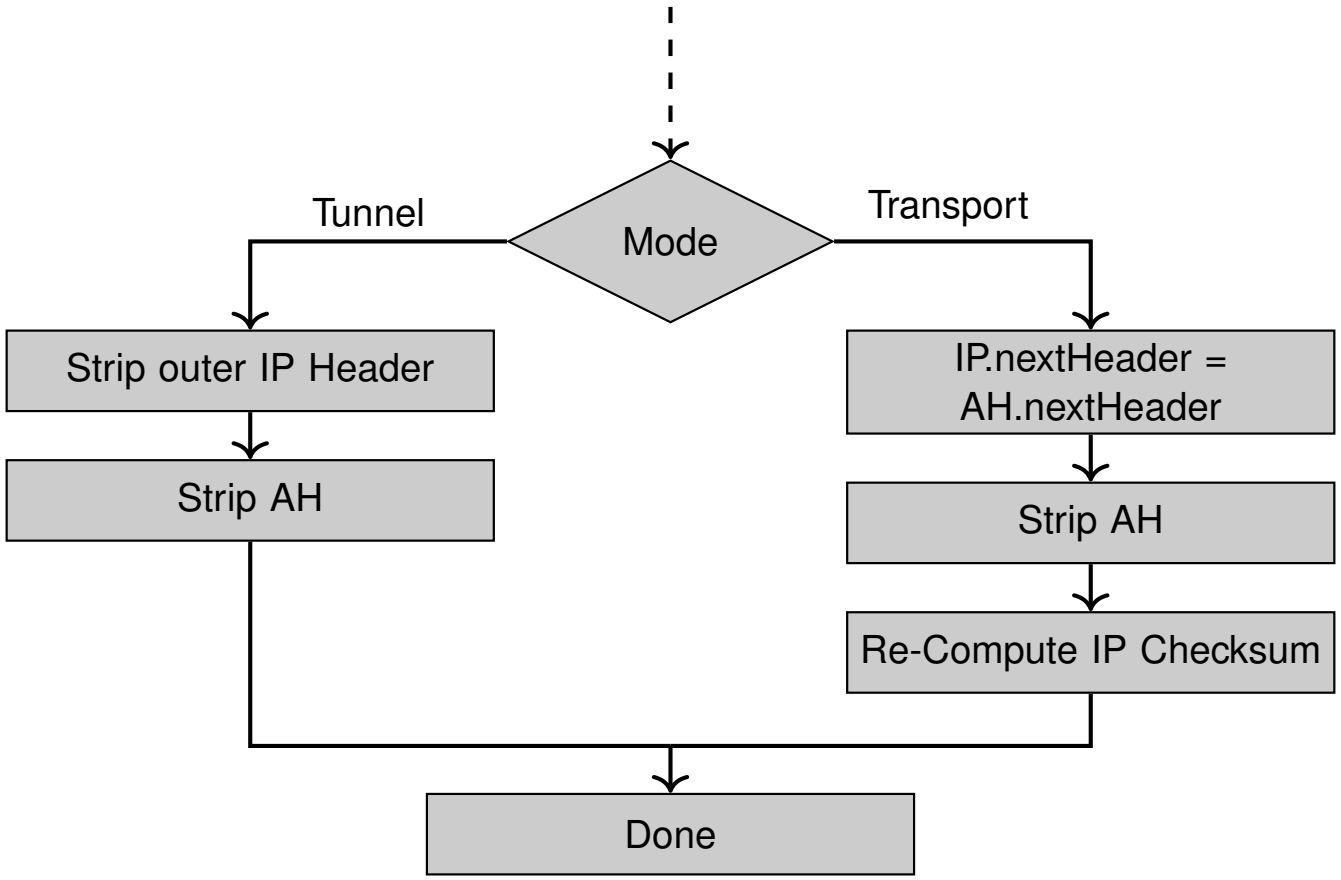




Inbound Processing



Inbound Processing



Cryptographic Algorithms in IPSec

Requirements on IPSec implementations are classified into different categories in [RFC4305]

ESP implementation requirements:

Requirement	Encryption Algorithm	Reference	Notes
MUST	NULL		1
MUST-	3DES-CBC	[RFC2451]	
SHOULD+	AES-CBC-128	[RFC3602]	
SHOULD	AES-CTR	[RFC3686]	
SHOULD NOT	DES-CBC	[RFC2405]	2

Requirement	Authentication Algorithm	Reference	Notes
MUST	NULL		1
MUST	HMAC-SHA1-96	[RFC2404]	3
SHOULD+	AES-XCBC-MAC-96	[RFC3566]	3
MAY	HMAC-MD5-96	[RFC2403]	3, 4

1. ESP encryption and auth are optional.
   - Support for the NULL algorithms required
   - Maintains consistency with service negotiation
   - Each may be NULL, both MUST NOT be NULL
2. DES provides only questionable security for general use nowadays
3. Designated "-96" in the Auth Algorithms
   - Output of the hash function is truncated
   - Truncation to 96 leftmost bits
4. Known weaknesses in MD5
   - Weaknesses should not affect use of MD5 with HMAC

Same algorithms for AH, except that NULL is not allowed

Chapter 12: IPSec

Introduction

IPSec Architecture

IPSec Security Protocols

Entity Authentication and Key Exchange

- Introduction
- Protocol Exchanges
- Generation of Keying Material
- Negotiation of Security Associations
- Advanced IKEv2 Features
- Message and Payload Format
- Summary
- Outlook

WireGuard

Introduction

Recap

Prior to any packet being protected, a *Security Association (SA)* has to be established between the two cryptographic endpoints.

SA establishment can be realized

- Manually
  - Does not scale well in large networks
  - Keys need to be distributed confidentially
- Dynamically
  - Using a standardized authentication & key management protocol
  - The *Internet Key Exchange protocol (IKE)* is such an architecture

Popular Implementation: IKEv2

- Standardized in [RFC4306] in December 2005
- Parts of IKEv1 poorly specified and spread over multiple RFCs
- IKEv2 provides unified authentication and key establishment
- Tries to achieve trade-off between features, complexity and security under realistic threat model
- [RFC5996] obsoletes [RFC4306]

$\Rightarrow$ In the following we will restrict our discussion to IKEv2

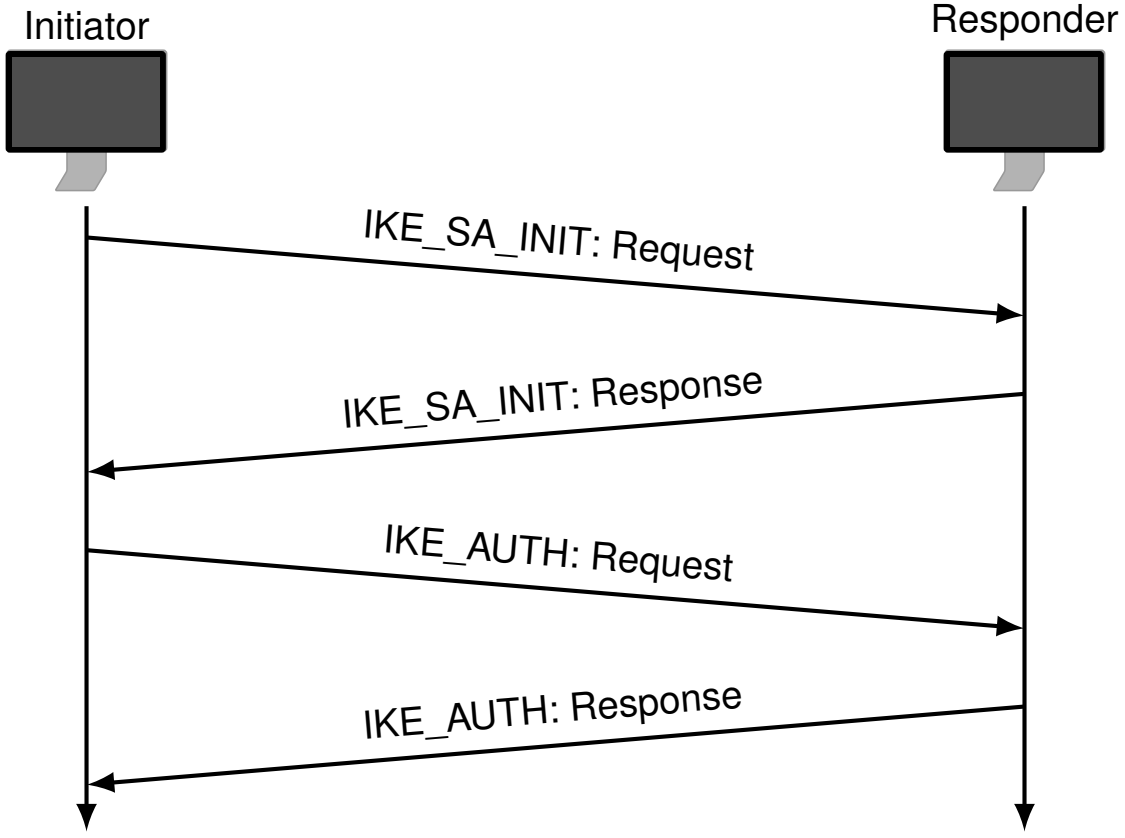
IKEv2 General Properties:

- Runs on UDP ports $\{500, 4500\}$
- Mutual authentication of the *Initiator* and *Responder*
- Negotiation of cryptographic suites (a complete set of algorithms used for SAs)
- Support for DoS mitigation through use of cookies
- Integrated support for requesting an IP address (useful for VPNs)
- IKEv2's latency is 2 round trips (i.e. 4 messages) in the common case

Protocol Exchanges

Basic Structure

- IKEv2 communication consists of message pairs
- Request and response
- One pair (request, response) is called an *exchange*
- An IKEv2 protocol run starts with two exchanges
  - IKE_SA_INIT
  - IKE_AUTH



Protocol Exchanges

IKE_SA_INIT

- Negotiates security parameters for a security association (IKE_SA)
- Sends nonces and Diffie-Hellman values
- IKE_SA is a set of security associations for protection of remaining IKE exchanges

IKE_AUTH

- Authenticates the previous messages
- Transmits identities
- Proves knowledge of corresponding identity secrets
- Creates first CHILD_SA
  - A CHILD_SA is a set of SAs, used to protect data with AH/ESP
  - The term *CHILD_SA* is synonymous to the common definition of an SA for IPSec AH and ESP

Protocol Exchanges

Exchanges:

IKE_SA_INIT and IKE_AUTH have to be completed in this strict order before any other exchanges

Additional IKEv2 exchanges might be:

- CREATE_CHILD_SA
  - Used to create another CHILD_SA
  - Can also be used for re-keying
- INFORMATIONAL
  - Keep-Alive
  - Deleting an SA
  - Reporting error conditions, ...

Remarks:

- IKE_AUTH includes the negotiation of a CHILD_SA to reduce the latency of IKEv2
- IKEv2 uses UDP as transport protocol
  - Requester must ensure reliability
  - E.g. retransmit request after timeout

Protocol Exchanges

Definition:

- $HDR(A, B)$:= IKEv2 message header with initiator SPI A and responder SPI B
- Protocol participants are called *Initiator* and *Responder*
- "[...]" marks optional components

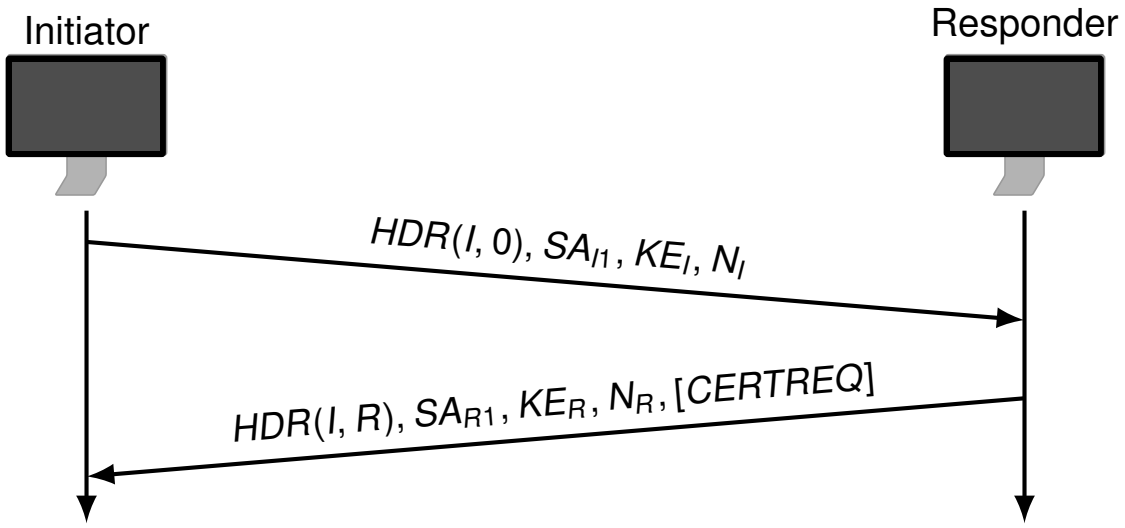
Message Payloads:

An IKEv2 message consists of the message header $HDR(A, B)$ and one or more *payloads*.

Next Payload Type	Notation	Value
No Next Payload		0
Security Association	SA	33
Key Exchange	KE	34
Identification - Initiator	IDi	35
Identification - Responder	IDr	36
Certificate	CERT	37
Certificate Request	CERTREQ	38
Authentication	AUTH	39
Nonce	Ni, Nr	40
Notify	N	41
Traffic Selector - Initiator	TSi	44
Traffic Selector - Responder	TSr	45
Configuration	CP	47

Other payloads are not addressed in this chapter and can be found in [RFC5996].

IKE_SA_INIT Exchange



- SA_{I1}: The cryptographic suites which the Initiator supports
- SA_{R1}: Cryptographic suite, chosen by Responder from SA_{I1}
- KE_I, KE_R: Public Diffie-Hellman values of Initiator and Responder
- N_I, N_R: Nonces of Initiator and Responder
- [*CERTREQ*]: (optional) Certificate request payload which is used for authentication via certificates

How are the encryption and authentication keys derived?

Generation of Keying Material

Algorithms:
In the context of an IKE_SA, 4 cryptographic algorithms are negotiated

- Encryption algorithmn
- Integrity protection algorithm
- Diffie-Hellman group (i.e. DH parameters p and g)
- Pseudo-Random function prf

Definition:

- $prf(K, S)$:= Pseudo-Random function with parameters K (Key) and S (Seed)
- e.g. $prf(K, S) = HMAC_K(S) = H(K \oplus opad \mid H(K \oplus ipad \mid S))$

Key Generation:

- The negotiated prf is used for construction of all subsequent keying material for IKE_SA and CHILD_SAs
- Amount of needed keying material may be greater than the output size of prf
- prf is used iteratively:

$$\begin{aligned}
\widetilde{prf}(K, S) &:= T_1 | T_2 | T_3 | ... \\
T_1 &:= prf(K, S \mid 0x01) \\
T_2 &:= prf(K, T_1 \mid S \mid 0x02) \\
T_3 &:= prf(K, T_2 \mid S \mid 0x03)
\end{aligned}$$

Negotiation of Security Associations

IKE_SA_INIT

- DH handshake in IKE_SA_INIT enables generation of a shared secret

$$SKEYSEED := prf(N_I \mid N_R, g_{IR})$$

- Each peer generates 7 different keys for different purposes

$$SK_d \mid SK_{AI} \mid SK_{AR} \mid SK_{EI} \mid SK_{ER} \mid SK_{PI} \mid SK_{PR} = \widetilde{prf}(SKEYSEED, N_I \mid N_R \mid SPI_I \mid SPI_R)$$

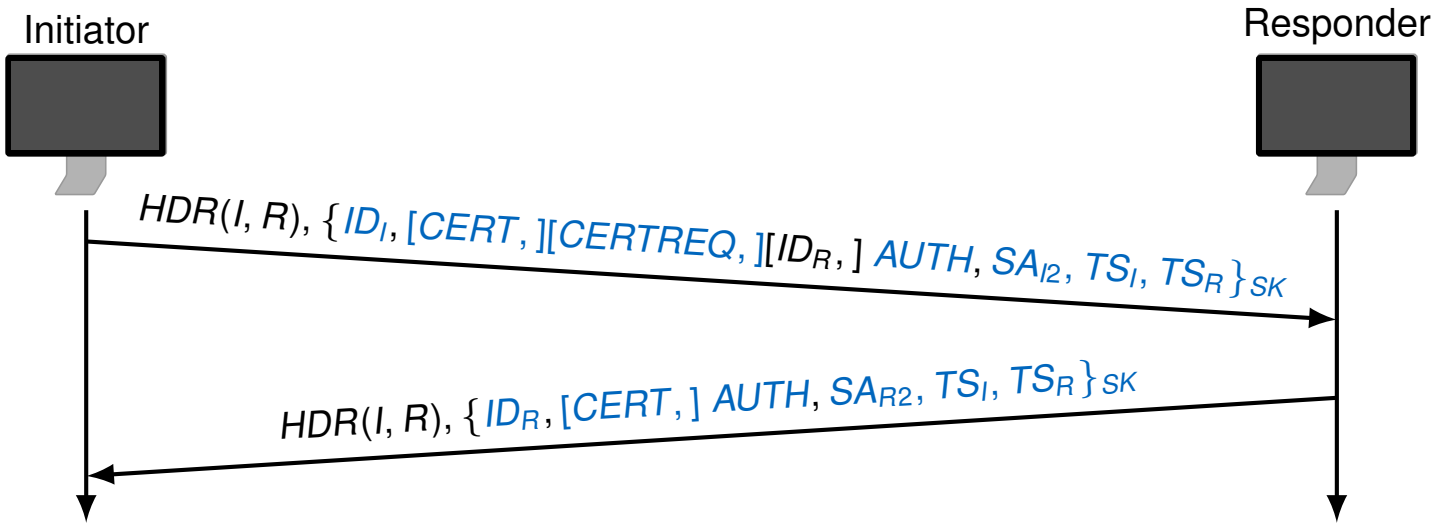
  - SK_D: Deriving new keys for the CHILD_SAs of this IKE_SA
  - SK_{AI}, SK_{AR}: Integrity protection of subsequent IKEv2 exchanges
  - SK_{EI}, SK_{ER}: Encryption of messages in subsequent IKEv2 exchanges
  - SK_{PI}, SK_{PR}: Generation of AUTH payloads for IKE_AUTH exchange

Current State:

- Now after the first exchange
  - the DH handshake is complete
  - keying material for the IKE_SA is generated
  - BUT: *SKEYSEED* and all derived keys are still *unauthenticated*
- Both parties now need to verify the identity of the other side
- Done in upcoming IKE_AUTH exchange
- All but the following IKEv2 headers are encrypted and integrity protected

IKE_AUTH Exchange

Let $\{\cdot\}_{SK}$ indicate encryption and integrity protection using that direction's SK_E and SK_A. Then



- Initiator and Responder assert their identities in the ID_I/ID_R payloads
- Initiator and Responder proove knowledge of the secrets corresponding to ID_I/ID_R
- They protect the integrity of the 1st/2nd message in the IKE_SA_INIT exchange, using the AUTH payload
- The Initiator may include a certificate or certificate chain and certificate request, if authentication should be conducted using digital signatures
- The remaining payloads $\{SA_{I2}, SA_{R2}, TS_I, TS_R\}$ are used to establish the first CHILD_SA

Authentication

- IKEv2 supports authentication via public key signatures or long-term pre-shared secret
- Authentication by signing (or calculating MAC_K of) a block of data
- The resulting value (*AUTH*) is transmitted in the IKE_AUTH exchange
- Authentication is conducted by verifying the validity of the received *AUTH* payload

The Initiator signs

$$(\text{IKE_SA_INIT request payload} \mid N_R \mid prf(SK_{PI}, \overline{ID}_I))$$

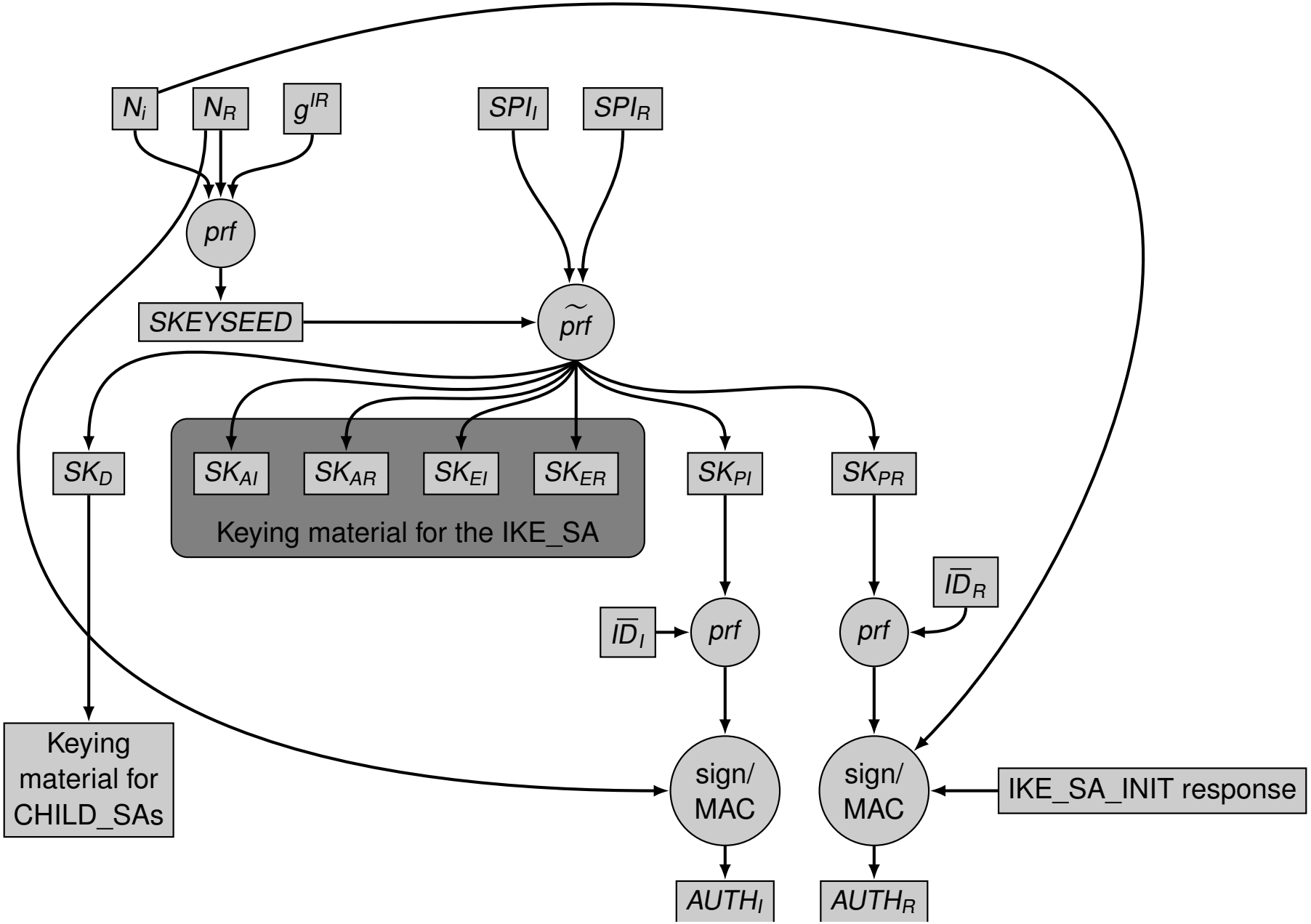
The Responder signs

$$(\text{IKE_SA_INIT response payload} \mid N_I \mid prf(SK_{PR}, \overline{ID}_R))$$

where

- *prf* is the algorithm negotiated in the IKE_SA_INIT exchange
- $\overline{ID}_I$, $\overline{ID}_R$ are the entire ID payloads excluding fixed headers (i.e. the payload header)

Negotiation of Security Associations



Authentication

- Initiator and Responder assert their identity by including $\overline{ID}_I$, $\overline{ID}_R$ in the *AUTH* payload
- It is important to include the other side's fresh N_I, N_R in *AUTH* to guarentee liveness and prevent replays
- The entire IKE_SA_INIT request/response is included in the *AUTH* generation $\Rightarrow$ Integrity of exchange
- In particular this guarantees that the proposed algorithms (SA_{I1}, SA_{R1}) and DH values (KE_I, KE_R) have not been modified by an attacker
- SK_{PI}, SK_{PR} are derived from *SKEYSEED* and involved in the generation of *AUTH*
  $\Rightarrow$ Every peer can verify that the freshly generated *SKEYSEED* is correct.

Authentication

In some cases, the Initiator may uses a MAC to generate the *AUTH* payload while the Responder uses a digital signature to generate the *AUTH* payload.

This could be the case, if the initiator is a user's host, connecting to an IPSec-based VPN server where

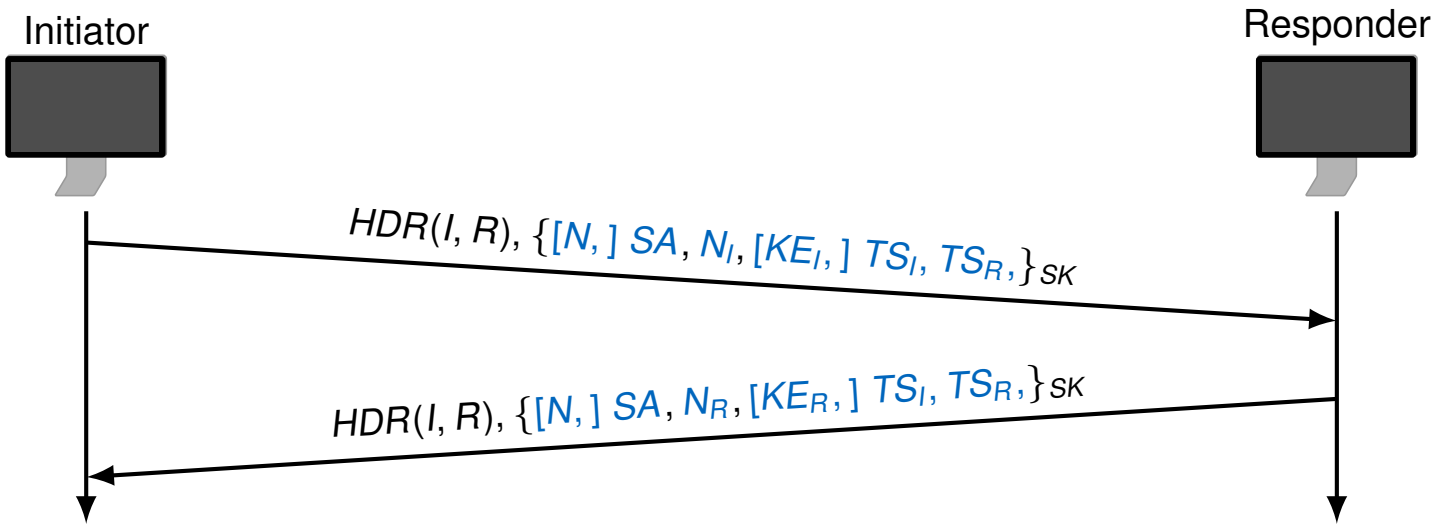
- the user's host is authenticated based on the user's password (or a key from a PBKDF)
- the VPN gateway is authenticated using its certificate

In case authentication is based on a certificate, IKEv2 does not enforce or suggest any algorithm for the digital signature. The certificate determines the algorithm used for the signature (e.g. RSA, DSA [El-Gamal], ECC)

Negotiation of Security Associations

CREATE_CHILD_SA Exchange

This exchange may be initiated by either end of the IKE_SA after the initial exchanges are completed



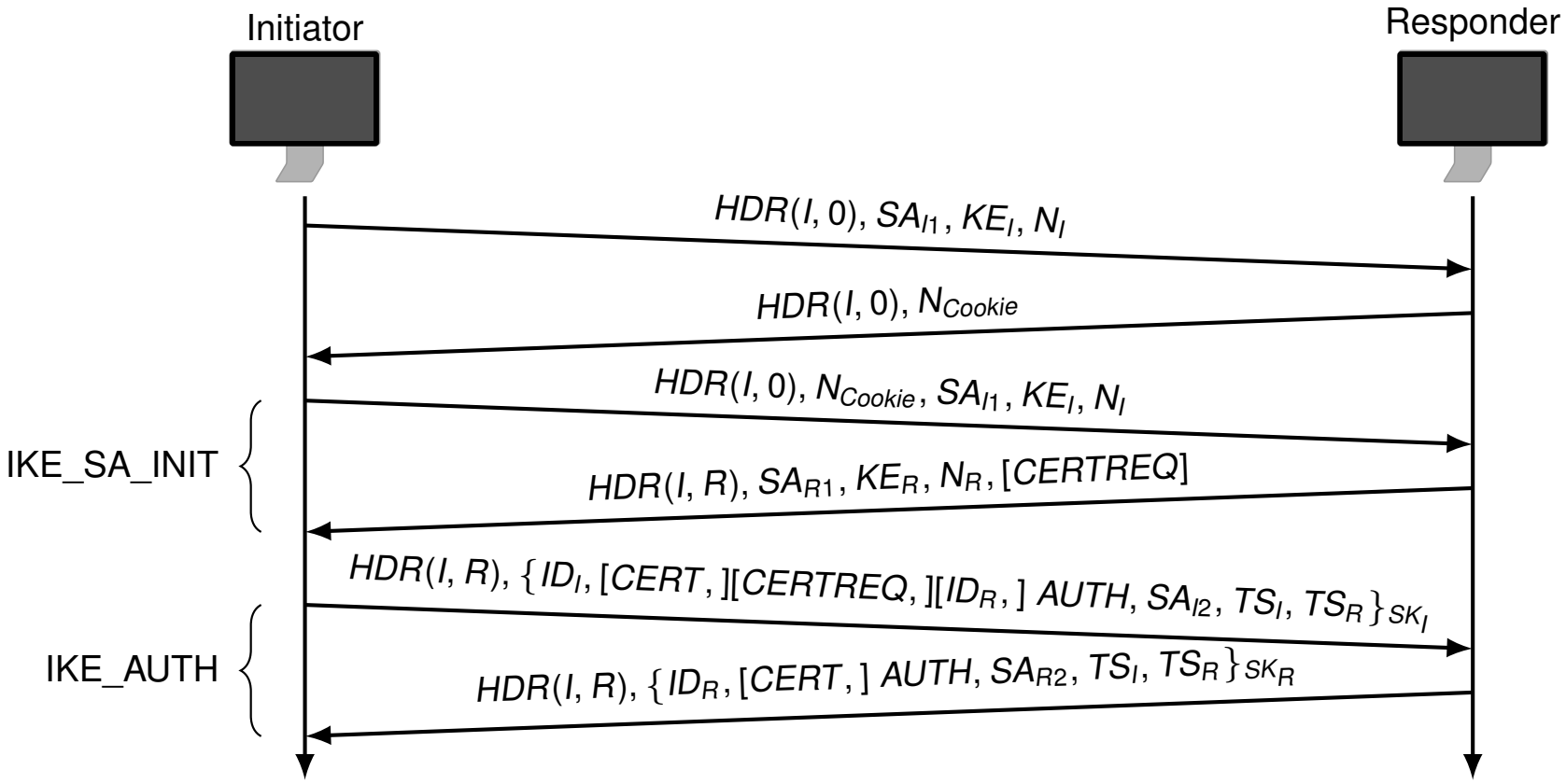
- N: Notify payload, only used for re-keying
- SA: SA offer(s)
- N_I, N_R: Nonces
- KE_I, KE_R: Optional DH values
- TS_I, TS_R: Proposed traffic selector payloads

- A single (CREATE_)CHILD_SA negotiation may result in multiple security associations
- Requesting an internal address and other configuration information on a remote network can be done by including a $CP_{Request}$ field
- The IPSec Remote Access Server (e.g. IPSec-based VPN gateway) may manage these addresses

Note: Don't confuse IKE SPI's in the *HDR* with CHILD_SA SPI's in the SA fields!

Protection against flooding attacks

- Flooding *R* with IKE_SA_INIT requests may result in state and CPU exhaustion
- *R* may switch to an alternative protocol with 3 exchanges if there are many *half-open* IKE_SAs
- In the alternative protocol, a cookie is exchanged *before* any state is preserved or CPU-intensive operations are conducted



Rekeying

Secret keys should only be used for a limited amount of time and to protect a limited amount of data.

Two possible solutions in IKEv2:

1. Establish new Security Associations
2. Re-establishment of Security Associations by *Rekeying*
    - Optional feature
    - Usage of CREATE_CHILD_SA exchange

Note:
Each endpoint is responsible for enforcing its own lifetime policy on SAs

Advanced IKEv2 Features

Traffic Selector Negotiation (TSN)

Traffic Selector (TS) payloads allow endpoints to communicate some of their Security Policy Database (SPD) information to their peers.

- New feature in IKEv2
- May be dynamically updated
- May serve as consistency check of SPDs

Each TS contains the following information:

- Address Range (IPv4 or IPv6)
- Port Range
- IP protocol ID

INFORMATIONAL Exchange

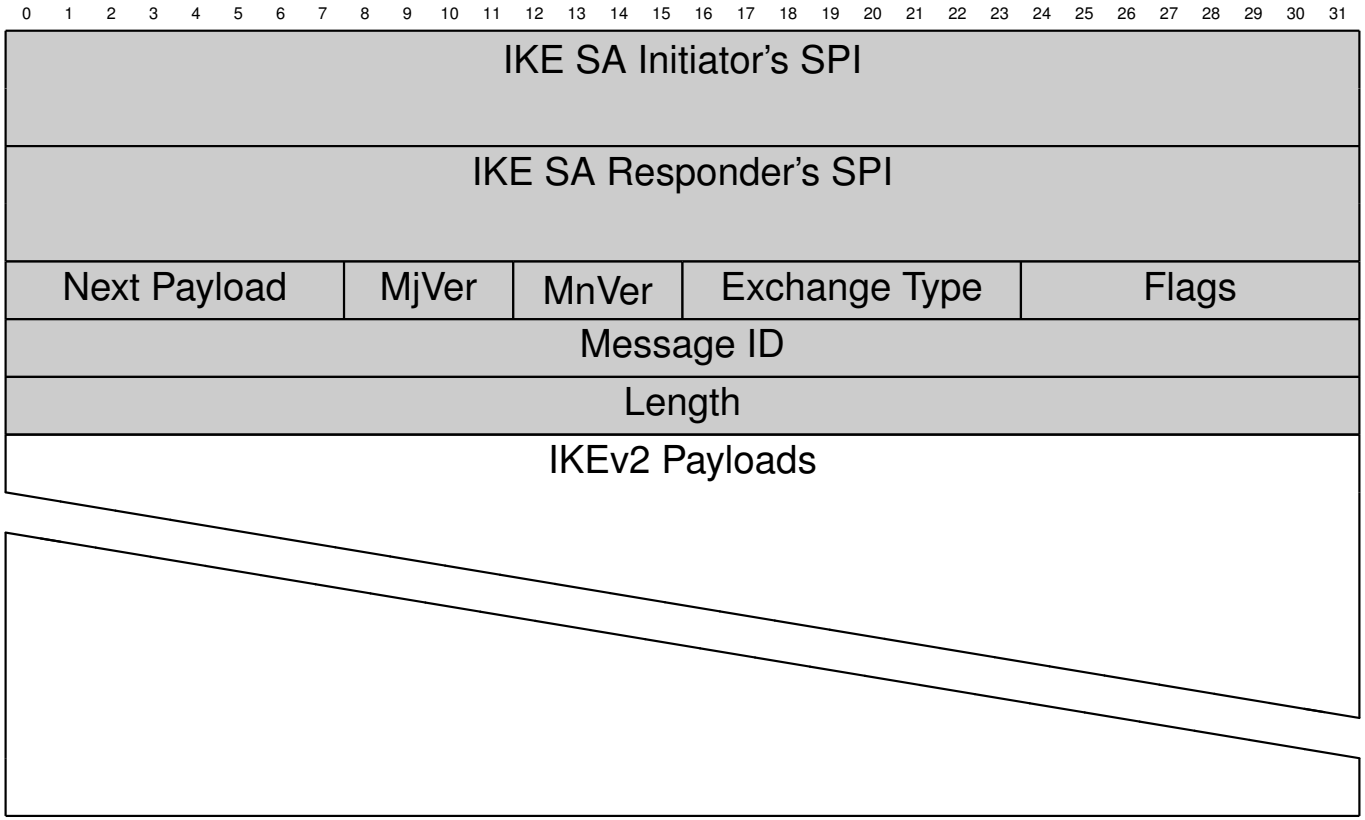
Used for sending

- Control messages (e.g. delete SA, configuration, ...)
- Error messages
- Notifications of certain events
- Keep-alive messages (empty payload)

INFORMATIONAL messages must only occur after initialization and are protected with the respective IKE_SA

Message and Payload Format

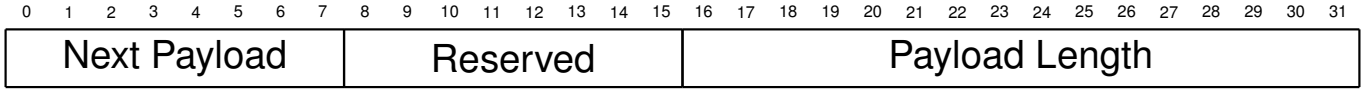
IKEv2 Header



- SA SPIs
  - Unlike ESP/AH both SPIs appear in header
  - In first message responder's SPI set to zero
- Next Payload (Code)
- Supported IKE versions (Major,Minor)
- Exchange Type (e.g. IKE_SA_INIT = 34)
- Flags
- Message ID (Sequence number)
- Length

IKEv2 Payload Header

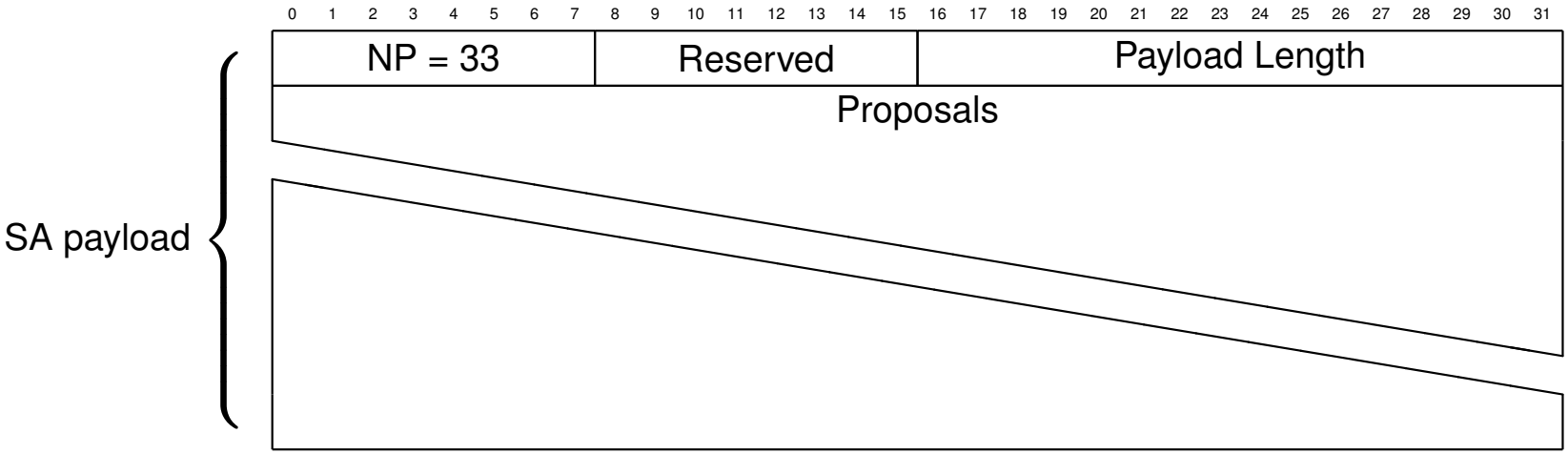
All IKEv2 payloads start with a common payload header:



- Next Payload
  - Payload type of the following payload
  - Same codes as in IKEv2 Header (e.g. 0 = no payload)
- Payload length
  - Total length of current payload
  - Includes this header

SA Negotiation (1)

An SA payload may contain multiple *proposals*. Each proposal contains a set of security protocols (IKEv2, AH, ESP) with the corresponding algorithms to be used.



Allowed proposal types:

- AH: Integrity check algorithms
- ESP: Encryption and integrity check algorithm
- IKEv2: Diffie-Hellman group, PRF algorithm, encryption and integrity check algorithm

SA Negotiation (2)

Proposals are ordered from most preferred to least preferred. Each proposal is identified with a proposal number (PN) such that

- Each proposal substructure has the same PN as the previous one or one greater
- The first proposal *must* have PN = 1
- If two successive proposal structures have the same PN, the proposal consists of both structures
- E.g. a proposal of AH *and* ESP would have two proposal substructures (one for AH and one for ESP)

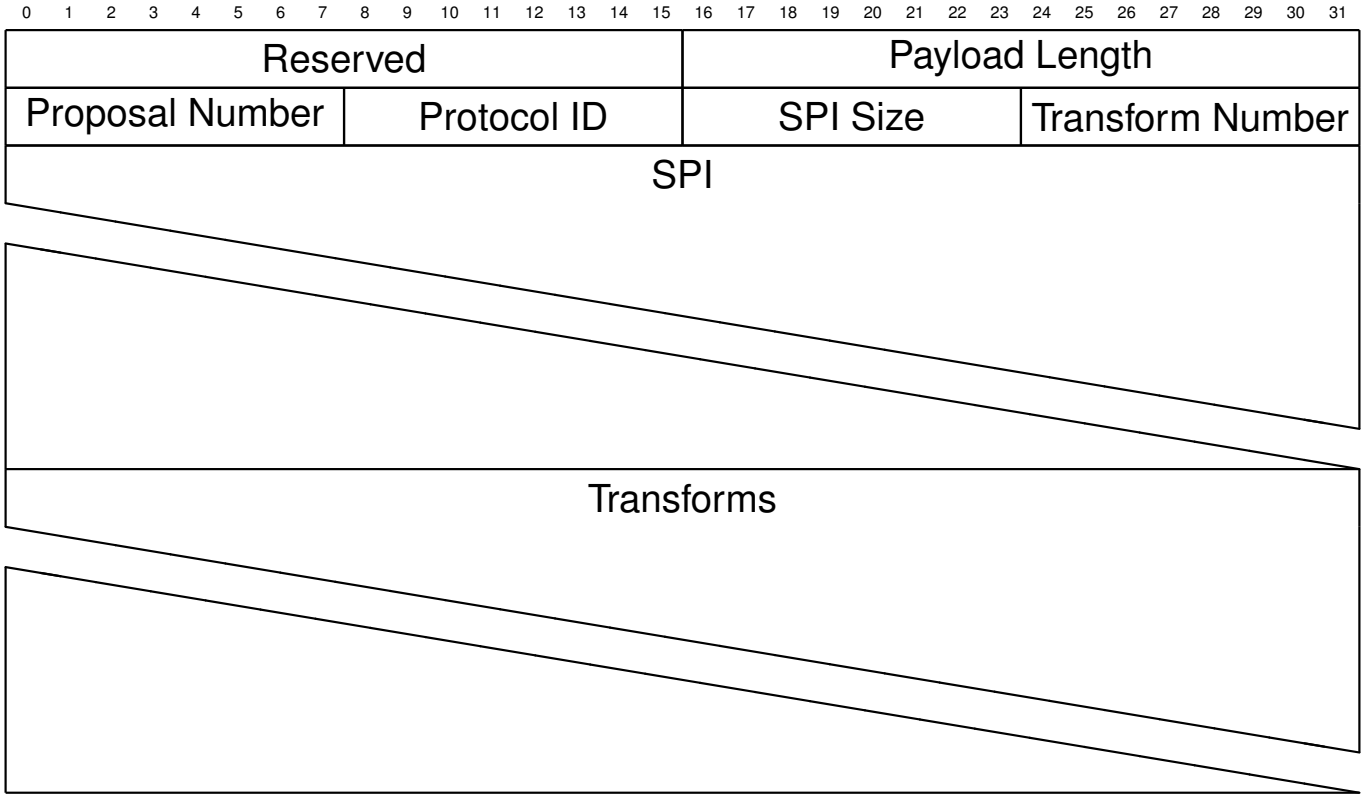
Each proposal substructure contains one or more *transforms*. Each transform specifies a cryptographic algorithm and may contain additional *attributes* that are required for the respective algorithm.

Thus Security Associations are encoded in a hierarchical manner:

$$\text{SA payload} \rightarrow \text{Proposals} \rightarrow \text{Protocols} \rightarrow \text{Transforms} \rightarrow \text{Attributes}$$

Message and Payload Format

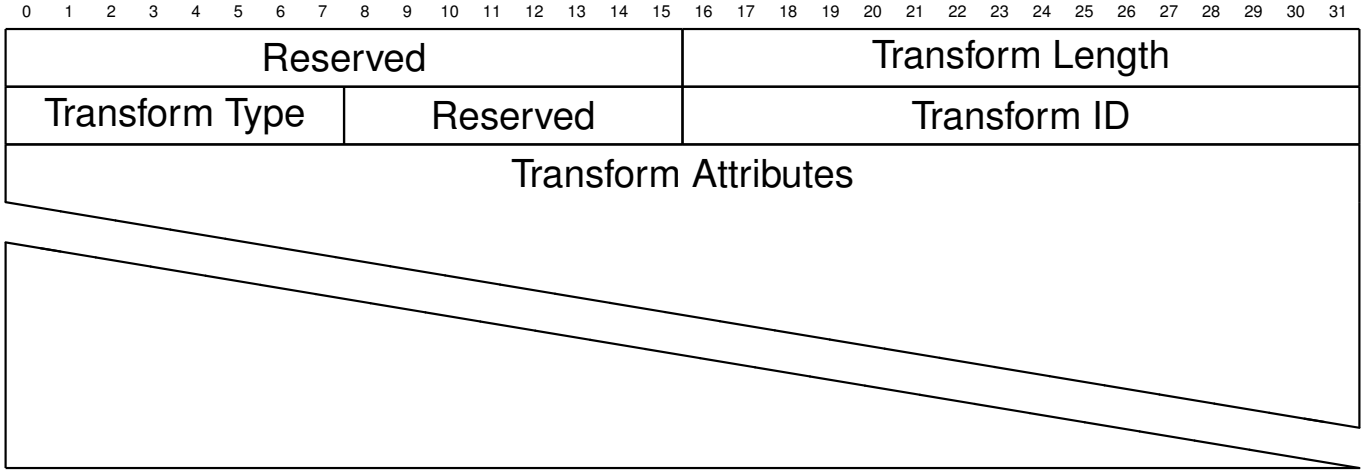
Proposal Structure



- Proposal Number
- Protocol ID (e.g. AH, ESP, ...)
- SPI Size

- Transform Number
  - How many transforms belong to this proposal
  - e.g. ESP may have multiple transforms
  - Request may have many transforms
  - Response must contain unique algorithm for each purpose

Transform Structure



Transform types:

- Integrity check algorithm (INTEG)
- Encryption algorithm (ENCR)
- Diffie-Hellman group (D-H)
- PRF algorithm (PRF)

Each transform is uniquley identified by an ID:

- IDs are listed in [RFC5996] Section 3.3.2
- e.g. ENCR_AES_CTR, PRF_HAMC_MD5

SA Negotiation (3)

Responders must select a single proposal from the ones proposed in the initiator's request (or reject all offers if none are acceptable).

Each of the proposal substructures in the response must contain a single transform of each transform type.

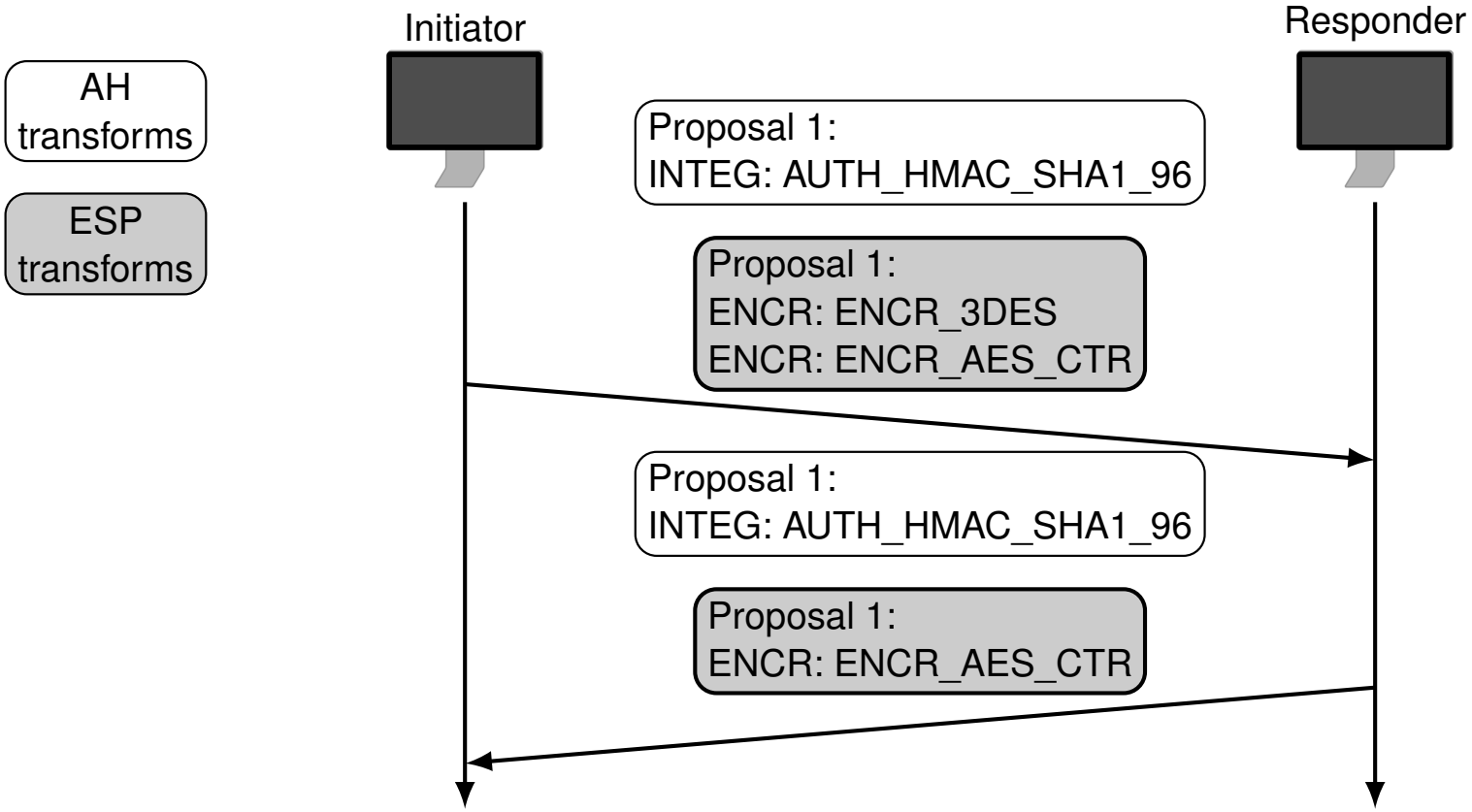
The responder should retain the Proposal Number field in the proposal payload

- Retention of proposal numbers should speed up initiator's protocol processing
- Avoids the need to compare responder's selection with every offered option
- Value enables intiator to compare directly and quickly

The initiator must verify that the SA payload received from the responder matches one of the initially sent proposals.

SA Negotiation (Example)

This example shows an ESP *and* AH protection suite.



1. The Initiator proposes one integrity transform and two encryption transforms
2. The Responder chooses at least one transform from every category
3. The Responder transmits his choice to the Initiator

Note:

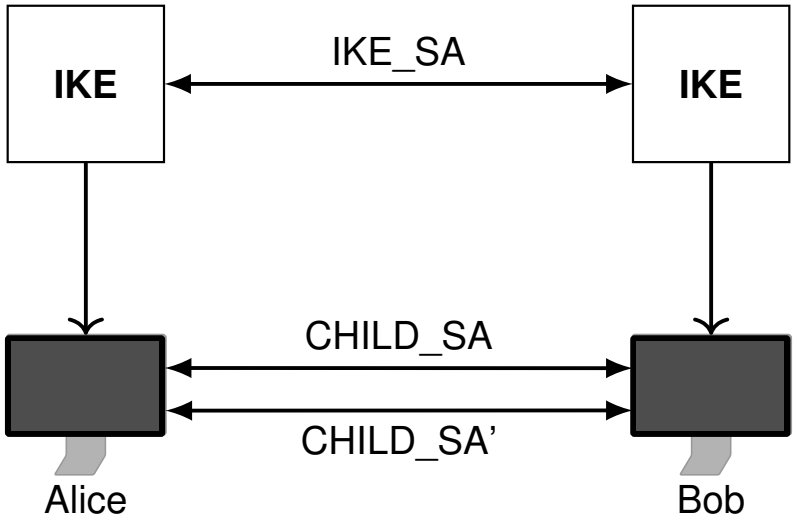
- Two or more structures with the same PN refer to the same proposal
- The SA negotiation in this example results in 4 SAs (2 per direction)

Summary

IPSec is IETF's security architecture for the Internet Protocol

- It provides to IP packets
  - Data origin authentication
  - Replay protection
  - Confidentiality
- Can be realized in intermediate or end systems
- Two fundamental security protcols are defined
  - Authentication Header (AH)
  - Encapsulating Security Payload (ESP)
- Both protocols can be run in
  - Transport Mode
  - Tunnel Mode
- Authentication, SA negotiation and Key management can be realized with IKEv2
- Authentication can be performed based on
  - Certificates or
  - Long-term pre-shared keys
- IKEv2 provides DoS protection using cookies

Summary



- IKE_SA is a set of Security Associations established after the initial IKEv2 exchange (IKE_SA_INIT)
- IKE_SA is used to encrypt and integrity protect all the remaining IKEv2 exchanges
- CHILD_SA is a set of Security Associations used to protect IP traffic with the AH/ESP protocol
- AH provides data integrity and replay protection
- ESP provides data integrity, replay protection and encryption